Aplicatie WEB pentru optimizarea calatoriilor pe baza criteriilor personalizate ale utilizatorului

Documentul de proiectare

Cuprins

1. Introducere 1

1.1 Scopul documentului 1

2. Prezentare generală și abordări de proiectare 2

2.1 Prezentare generală 2
2.2 Presupuneri/ Constrângeri/ Riscuri 2
2.2.1 Presupuneri 2
2.2.2 Constrângeri 2
2.2.3 Riscuri 3

3. Considerațiii de proiectare 4

3.1 Obiective și linii directoare (ghiduri) 4
3.2 Metode de dezvoltare 4
3.3 Strategii de arhitectură 4

4. Arhitectura Sistemului și Proiectarea Arhitecturii 6

4.1 Vedere logică 6
4.2 Arhitectură hardware 6
4.3 Arhitectură software 6
4.4 Arhitectura informațiilor 7
4.5 Arhitectura de comunicații interne 7
4.6 Diagrama de arhitectură a sistemului 8

5. Proiectarea sistemului 9

5.1 Proiectarea bazei de date 9
5.1.1 Obiecte de date și structuri de date rezultante 9
5.1.2 Fișiere și baze de date 9
5.2 Conversii de date 9
5.3 Interfețe utilizator 10
5.3.1 Intrări 10
5.3.2 Ieșiri 10
5.4 Proiectarea interfețelor cu utilizatorul 10

6. Scenarii de utilizare 11

7. Proiectare de detaliu 12

7.1 Proiectare hardware de detaliu 12
7.2 Proiectare software de deatliu 12
7.3 Proiectare detaliată de securitate 13
7.4 Proiectare de detaliu pentru performanța sistemului 14
7.5 Proiectare detaliată a comunicațiilor interne (între componente) 14

8. Controale pentru verificarea integrității sistemului 15

Anexa A: Gestiunea modificărilor documentului 16

Anexa B: Acronime 17

Anexa C Documente la care se face referire 18

1. Introducere

Prezentul document descrie proiectarea tehnică a platformei TripSmart Agency, o aplicație web dezvoltată ca lucrare de licență sub titlul "Aplicație WEB pentru optimizarea călătoriilor pe baza criteriilor personalizate ale utilizatorului". Scopul acestei aplicații este de a oferi utilizatorilor un instrument inteligent de planificare a călătoriilor, care să permită personalizarea rutelor în funcție de priorități precum timpul minim, costul minim, evitarea traficului sau distanța minimă.

Numele complet al proiectului este TripSmart Agency, iar acronimul utilizat în cadrul documentației este TSA. Sistemul propus este o soluție originală, dezvoltată integral de la zero în cadrul pregătirii lucrării de licență, neexistând un sistem anterior care să fie înlocuit sau modificat. Platforma se adresează atât turiștilor individuali care doresc să își planifice călătoriile, cât și agențiilor de turism care doresc să ofere clienților lor servicii avansate de planificare.

Evoluția așteptată a acestui document este una incrementală, pe măsură ce dezvoltarea aplicației avansează. Pe parcursul implementării, pot apărea decizii de proiectare care ajustează arhitectura inițială, iar documentul va fi actualizat corespunzător pentru a reflecta starea curentă a sistemului. Versiunea finală a documentului va fi sincronizată cu versiunea finală a aplicației, în momentul depunerii lucrării de licență.

În ceea ce privește securitatea și confidențialitatea asociate cu utilizarea acestui document, se menționează următoarele aspecte. Documentul conține informații detaliate despre arhitectura sistemului, structura bazei de date, endpoint-urile API și mecanismele de autentificare. Aceste informații sunt considerate confidențiale în contextul evaluării lucrării de licență și nu vor fi distribuite public. Accesul la acest document este restricționat la următoarele categorii de persoane: studentul autor al lucrării, coordonatorul științific și membrii comisiei de evaluare. Documentul nu va fi publicat online sau partajat cu terți în afara contextului academic. Informațiile sensibile, precum cheile de criptare, parolele implicite pentru baza de date sau datele personale ale utilizatorilor utilizați în teste, nu sunt incluse în acest document, ci sunt gestionate separat prin fișiere de configurare care nu fac parte din documentația publică.

Documentul de Proiectare a Sistemului are rolul de a transforma cerințele funcționale și non-funcționale, stabilite în prealabil în Documentul de Specificare a Cerințelor, în specificații tehnice concrete. Acesta descrie modul în care sistemul va fi construit, ce componente hardware și software vor fi utilizate, cum vor comunica între ele aceste componente și cum vor fi implementate funcționalitățile cerute. Se documentează proiectarea sistemului atât la un nivel înalt de abstractizare, cât și la nivel de detaliu, oferind astfel o înțelegere completă a soluției tehnice propuse.

Se descriu obiectivele și considerațiile care au ghidat proiectarea, se furnizează o prezentare generală la nivel înalt a arhitecturii sistemului, inclusiv componentele hardware, software și de comunicații, și se detaliază proiectarea bazei de date, a interfeței cu utilizatorul și a scenariilor operaționale principale.

Proiectarea sistemului la nivel înalt este descompusă în continuare în specificații detaliate pentru fiecare componentă. Astfel, se vor descrie în detaliu componentele hardware necesare, protocoalele și canalele de comunicații interne între componente, modulele software cu responsabilitățile și interfețele lor, controalele de integritate care asigură funcționarea corectă a sistemului și interfețele externe cu care sistemul interacționează.

1.1 Scopul documentului

Scopul principal al acestui document este de a oferi o specificație tehnică completă și coerentă a sistemului TripSmart Agency, care să servească drept ghid de implementare pe parcursul dezvoltării aplicației. Prin acest livrabil se documentează și se urmăresc informațiile necesare pentru a defini eficient arhitectura și designul sistemului, în scopul de a oferi echipei de dezvoltare îndrumare asupra arhitecturii sistemului ce urmează să fie dezvoltat. În contextul acestui proiect de licență, echipa de dezvoltare este reprezentată de un singur dezvoltator, studentul autor, dar principiile și structura documentului respectă standardele industriei.

Documentele de proiectare sunt produse incremental și iterativ pe parcursul ciclului de viață al dezvoltării sistemului, în funcție de circumstanțele particulare ale proiectului de tehnologie informațională și de metodologia de dezvoltare a sistemului utilizată. Pentru TripSmart Agency, se utilizează o metodologie de dezvoltare în cascadă adaptată, cu iterații corespunzătoare fazelor de implementare definite inițial. Fiecare iterație completează un modul funcțional al aplicației, iar documentul de proiectare este actualizat pentru a reflecta deciziile luate și eventualele ajustări arhitecturale.

Publicul țintă al acestui document este format din trei categorii principale de cititori. Managerul de proiect, reprezentat de coordonatorul lucrării de licență, va utiliza acest document pentru a evalua corectitudinea soluției tehnice și pentru a verifica alinierea cu cerințele inițiale. Echipa de proiect și echipa de dezvoltare, reprezentate de studentul autor, vor folosi documentul ca referință zilnică în procesul de codare, testare și integrare a componentelor. Anumite părți ale acestui document, cum ar fi descrierea interfeței cu utilizatorul și scenariile de utilizare, pot fi împărtășite cu utilizatorii finali pentru validarea aspectelor vizuale și funcționale, precum și cu comisia de evaluare pentru verificarea coerenței și calității soluției propuse.

Scopul specific al acestui document este, așadar, să ofere o hartă clară a modului în care aplicația TripSmart Agency va fi construită, să detalieze toate deciziile tehnice, arhitecturale și de implementare, și să servească drept ghid de referință pe parcursul întregului proces de dezvoltare, de la prima linie de cod până la testarea finală și livrarea lucrării.

2. Prezentare generală și abordări de proiectare

Această secțiune descrie principiile și strategiile care vor fi utilizate ca ghiduri în momentul proiectării și implementării sistemului.

2.1 Prezentare generală

TripSmart Agency este o aplicație web de tip client-server, organizată pe o arhitectură pe trei niveluri. Primul nivel este stratul de prezentare, realizat cu HTML5, CSS3, JavaScript și integrarea hărții prin Leaflet.js. Al doilea nivel este stratul de logică de afaceri, implementat cu Node.js și framework-ul Express. Al treilea nivel este stratul de date, folosind PostgreSQL ca sistem de gestiune a bazelor de date.

Aplicația oferă funcționalități de planificare a rutelor cu optimizare multi-criterială, analiză a traficului, gestionare conturi utilizator și administrare. Inovația principală a proiectului constă în implementarea unui algoritm de optimizare bazat pe scoring ponderat, care permite utilizatorului să își personalizeze călătoria în funcție de prioritățile proprii precum timpul minim, costul minim, evitarea traficului sau distanța minimă.

2.2 Presupuneri/ Constrângeri/ Riscuri

2.2.1 Presupuneri

Proiectul se bazează pe următoarele presupuneri. Utilizatorii au acces la internet și un browser modern precum Chrome, Firefox sau Edge, deoarece aplicația este livrată exclusiv online. Datele de trafic vor fi inițial simulate, nu în timp real, din cauza costurilor și complexității accesului la date în timp real. API-ul OpenStreetMap va fi disponibil gratuit pe durata dezvoltării, fără restricții care să afecteze proiectul. Serverul de dezvoltare rulează pe localhost cu Node.js versiunea 18 sau superioară. Utilizatorii au cunoștințe minime de operare cu aplicații web și pot completa formulare, naviga între pagini și interpreta informațiile afișate pe hartă.

2.2.2 Constrângeri

Mediu hardware și software

Aplicația rulează pe un singur calculator (laptop/PC) folosit pentru dezvoltare și demonstrație. Hardware-ul minim recomandat este un procesor Intel Core i5, 8GB RAM și 256GB SSD. Sistemul de operare poate fi Windows 10/11 sau Ubuntu 20.04/22.04. Browser-ul trebuie să fie Chrome sau Firefox în versiuni recente. Impact: aplicația trebuie să fie ușoară, consum redus de resurse (maxim 512MB RAM pentru Node.js).

Mediu utilizator final

Utilizatorii au niveluri diferite de competență tehnică și accesează platforma de pe dispozitive cu rezoluții variate (de la mobil la desktop). Impact: interfața trebuie să fie intuitivă, responsive și accesibilă, cu mesaje de eroare clare și suport pentru navigare cu tastatura.

Disponibilitatea resurselor

Bugetul proiectului este zero. Toate tehnologiile și API-urile utilizate trebuie să fie gratuite și open-source. Impact: se folosește OpenStreetMap în loc de Google Maps (fără rutare automată), datele de trafic sunt generate sintetic (nu în timp real).

Conformitatea cu standardele

Aplicația trebuie să respecte standardele web: HTML5, CSS3, ECMAScript 2020+, convenții REST pentru API (coduri HTTP standard). Impact: compatibilitate cu toate browserele moderne.

Cerințe de interoperabilitate

Platforma interoperează doar cu API-ul extern OpenStreetMap. Impact: necesită implementarea de mecanisme de retry și mesaje de eroare pentru cazurile în care API-ul extern nu răspunde.

Cerințe de interfață și protocol

Comunicarea frontend-backend se face exclusiv prin HTTP și JSON. Formatele mesajelor sunt standardizate. Impact: simplitate și predictibilitate în dezvoltare.

Cerințe de licențiere

Toate componentele (Node.js, Express, PostgreSQL, Leaflet, Chart.js) au licențe open-source compatibile cu uzul academic (MIT, BSD, PostgreSQL). Impact: codul poate fi distribuit ca parte a lucrării de licență fără restricții.

Depozitarea și distribuția datelor

Datele utilizatorilor (nume, email, parolă criptată) sunt stocate local în baza de date PostgreSQL pe aceeași mașină. Nu există distribuție în cloud. Impact: backup-ul și securitatea sunt responsabilitatea dezvoltatorului.

Cerințe de securitate

Parolele sunt criptate cu bcrypt, toate interogările SQL sunt parametrizate pentru prevenirea SQL injection, sesiunile sunt gestionate securizat. Impact: protecție de bază împotriva vulnerabilităților comune.

Limitări ale capacității de memorie

Resursele sunt limitate la cele ale unui laptop standard. Impact: algoritmul de optimizare a rutelor este limitat la un graf cu maxim 50 de noduri (orașe) pentru a menține timpul de răspuns sub 3 secunde.

Cerințe de performanță

Timpul de răspuns pentru calculul rutei trebuie să fie sub 3 secunde. Impact: algoritmul trebuie optimizat, iar rezultatele pot fi stocate în cache pentru rute frecvente.

Comunicații de rețea

Toate componentele rulează pe aceeași mașină (localhost), fără trafic de rețea extern (excepție: API-ul OpenStreetMap). Impact: latențe minime, fără riscuri de securitate legate de rețea.

Verificare și validare (testare)

Testarea se face manual pentru UI și fluxuri funcționale, plus teste unitare în Jest pentru algoritmul de optimizare. Impact: acoperire limitată, focus pe funcționalitățile critice.

Obiective de calitate

Prioritățile sunt: corectitudinea algoritmului de optimizare > funcționalități de bază > securitate > performanță > estetică vizuală. Impact: dezvoltarea se concentrează mai întâi pe algoritm și funcțiile esențiale.

Alte cerințe din Documentul de Cerințe

Toate cerințele specificate în SRS trebuie respectate. Impact: modulul de administrare este opțional, dar implementarea lui aduce un plus la notă.

2.2.3 Riscuri

Risc 1: API-ul OpenStreetMap poate avea întreruperi

Descriere: API-ul extern de hărți folosit de aplicație poate fi temporar indisponibil sau poate răspunde lent, ceea ce afectează afișarea hărții și funcționalitatea de geocodare (transformarea numelor de locații în coordonate).

Strategie de reducere: Se implementează un mecanism de retry cu backoff exponențial pentru cererile eșuate. Se afișează mesaje de eroare prietenoase utilizatorului, precum "Serviciul de hărți

este temporar indisponibil. Vă rugăm să încercați din nou." Se oferă posibilitatea de a introduce manual coordonatele GPS ca alternativă.

Risc 2: Algoritmul de optimizare este prea lent pentru rute complexe

Descriere: Algoritmul de scoring ponderat care calculează ruta optimă poate deveni lent atunci când graful rutelor conține un număr mare de noduri (orașe) sau când sunt evaluate numeroase rute posibile. Acest lucru poate duce la depășirea pragului de 3 secunde stabilit pentru timpul de răspuns.

Strategie de reducere: Se limitează graful rutelor la maximum 50 de noduri (principalele orașe din România). Se implementează un sistem de caching care stochează rezultatele calculelor pentru rutele frecvente, folosind chei generate din perechile origine-destinație. Se optimizează codul algoritmului pentru a evita calculele redundante.

Risc 3: Dependențe Node.js cu vulnerabilități de securitate

Descriere: Pachetele npm utilizate în proiect (Express, bcrypt, pg etc.) pot avea vulnerabilități de securitate cunoscute, care ar putea fi exploatate în anumite scenarii.

Strategie de reducere: Se rulează periodic comanda npm audit pentru a identifica vulnerabilitățile cunoscute. Se actualizează pachetele la versiunile corectate, testând în prealabil că actualizările nu introduc probleme de compatibilitate. Se folosesc versiuni stabile și bine testate ale bibliotecilor, nu versiuni bleeding edge.

Risc 4: Integrarea hărții Leaflet are probleme pe anumite browsere

Descriere: Diferite browsere pot interpreta diferit JavaScript-ul sau pot avea suport limitat pentru WebGL și randarea hartelor, ceea ce poate duce la afișarea incorectă a hărții sau la erori de funcționare.

Strategie de reducere: Se testează aplicația pe toate browserele țintă: Google Chrome, Mozilla Firefox și Microsoft Edge în versiunile lor recente. Se adaugă mesaje de avertizare pentru browserele mai vechi sau neacceptate. Se folosesc fallback-uri acolo unde este posibil.

Risc 5: Datele simulate de trafic nu reflectă realitatea

Descriere: Deoarece datele de trafic sunt generate sintetic și nu provin de la o sursă reală, ele pot fi inexacte sau nerealiste, ceea ce reduce valoarea practică a recomandărilor oferite utilizatorilor.

Strategie de reducere: Se specifică clar în interfața utilizatorului, printr-un mesaj vizibil, că datele de trafic sunt demonstrative și nu trebuie folosite pentru decizii critice în timp real. Se documentează acest aspect în lucrarea de licență, subliniind caracterul demonstrativ al funcționalității. Se creează un set de date cât mai realist, bazat pe modele statistice simple.

Risc 6: Pierderea datelor din baza de date

Descriere: O defecțiune a hard disk-ului, o comandă greșită de ștergere sau o corupere a bazei de date poate duce la pierderea tuturor datelor utilizatorilor, călătoriilor și traficului.

Strategie de reducere: Se realizează backup-uri periodice ale bazei de date PostgreSQL folosind comanda pg_dump, stocate într-un director separat. Se păstrează scriptul de inițializare a bazei de date (schema.sql) în repository-ul Git, astfel încât o bază de date goală poate fi recreată rapid. Se exportă datele importante în format JSON la sfârșitul fazelor majore de dezvoltare.

Risc 7: Complexitatea implementării autentificării

Descriere: Implementarea corectă și securizată a autentificării utilizatorilor (înregistrare, login, sesiuni, criptare parole) poate fi mai complexă decât pare inițial, existând riscul unor greșeli care să compromită securitatea.

Strategie de reducere: Se folosesc biblioteci middleware standard și bine testate: express-session pentru gestionarea sesiunilor și bcrypt pentru criptarea parolelor. Nu se implementează manual funcții criptografice sau de gestionare a sesiunii. Se urmează tutoriale și bune practici din documentația oficială.

Risc 8: Neîndeplinirea termenului de predare

Descriere: Datorită complexității proiectului sau a unor dificultăți neprevăzute în implementare, există riscul ca toate modulele planificate să nu fie finalizate până la termenul limită de predare a lucrării de licență.

Strategie de reducere: Se prioritizează implementarea funcționalităților în funcție de importanța lor: algoritmul de optimizare și modulul de planificare sunt esențiale; autentificarea și salvarea în istoric sunt importante; modulul de administrare este opțional dar aduce plus la notă. Se alocă timp suficient pentru testare și documentare în planificarea săptămânală. Se comunică proactiv eventualele întârzieri coordonatorului

3. Considerațiii de proiectare

Această secțiune descrie problemele care trebuie abordate sau rezolvate înainte de a încerca să se elaboreze o soluție de design completă pentru platforma TripSmart Agency.

3.1 Obiective și linii directoare (ghiduri)

Obiectivele principale ale proiectului

Primul obiectiv este corectitudinea algoritmului de optimizare. Algoritmul multi-criterial de scoring ponderat reprezintă inovația centrală a lucrării de licență și trebuie să funcționeze corect în toate scenariile. Acesta trebuie să normalizeze corect valorile (timp, cost, trafic, distanță) și să aplice ponderile în funcție de criteriul selectat de utilizator. Prioritatea acestui obiectiv este maximă, deoarece fără el proiectul își pierde valoarea științifică.

Al doilea obiectiv este finalizarea funcționalităților de bază înaintea celor secundare. Modulul de planificare a rutei și algoritmul de optimizare sunt esențiale. Autentificarea și salvarea în istoric sunt importante pentru a demonstra un sistem complet. Modulul de administrare și predicțiile de trafic sunt opționale, dar aduc un plus la nota finală. Această prioritizare asigură că, în caz de întârziere, cel puțin nucleul proiectului este funcțional.

Al treilea obiectiv este performanța, cu un timp de răspuns sub 3 secunde pentru calculul rutei. Acest prag este considerat acceptabil pentru o aplicație web și asigură o experiență plăcută utilizatorului. Dacă timpul de răspuns depășește 5 secunde, utilizatorul poate percepe aplicația ca fiind lentă sau nefuncțională.

Al patrulea obiectiv este securitatea de bază. Parolele utilizatorilor trebuie criptate cu bcrypt înainte de stocarea în baza de date. Toate interogările SQL trebuie să fie parametrizate pentru a preveni atacurile de tip SQL injection. Sesiunile utilizatorilor trebuie gestionate securizat, iar resursele administrative trebuie protejate prin verificarea câmpului is_admin.

Al cincilea obiectiv este ușurința în întreținere. Codul trebuie să fie bine structurat, comentat acolo unde este necesar, și să urmeze convenții de denumire clare. Acest obiectiv este important pentru ca lucrarea de licență să fie ușor de evaluat și pentru ca viitorii dezvoltatori (inclusiv autorul, în cazul unor posibile extensii) să poată înțelege rapid logica aplicației.

Linii directoare de codare

Pentru frontend, se utilizează indentarea cu 2 spații, nu cu tab-uri, pentru a asigura o afișare consistentă în toate editorile. Numele funcțiilor și variabilelor sunt în format camelCase (de exemplu, calculateRoute, userEmail). Numele fișierelor sunt în format kebab-case (de exemplu, planificare.js, analiza-trafic.css). Fișierele sunt organizate pe categorii: toate paginile HTML în folderul pages, toate stilurile CSS în folderul css, toate scripturile JavaScript în folderul js, iar imaginile și alte resurse statice în folderul assets.

Pentru backend, se utilizează aceeași indentare cu 2 spații. Numele endpoint-urilor API sunt în format kebab-case (de exemplu, /api/user/login, /api/trips/optimize). Codul este organizat pe straturi separate în foldere distincte: routes pentru definirea endpoint-urilor, controllers pentru logica de procesare, models pentru interacțiunea cu baza de date, middleware pentru funcțiile de interceptare, services pentru logica de business complexă. Fiecare fișier are un singur scop și poartă un nume sugestiv.

Pentru baza de date, numele tabelelor sunt la plural și în snake_case (users, trips, traffic_data). Numele coloanelor sunt de asemenea în snake_case (user_id, created_at, congestion_level). Cheile primare se numesc id, iar cheile străine poartă numele tabelei referite urmate de _id (de exemplu, user_id în tabela trips). Această convenție este standard în ecosistemul PostgreSQL și face interogările mai ușor de scris și citit.

Politici și strategii de design fără implicații arhitecturale majore

În ceea ce privește alegerea produselor specifice, se folosește Leaflet.js versiunea 1.9.4 pentru hartă deoarece este open-source și se integrează bine cu OpenStreetMap. Nu se utilizează Google Maps deoarece necesită chei API și poate genera costuri. Pentru grafice, se folosește Chart.js versiunea 4.4.0 deoarece este ușor de utilizat și oferă tipurile de grafice necesare (linii pentru evoluția în timp, bare pentru comparații). Nu se utilizează D3.js deoarece este prea complex pentru nevoile proiectului.

Pentru gestionarea erorilor, se adoptă strategia de a afișa utilizatorului mesaje prietenoase și de a loga erorile în consola serverului pentru depanare. Pe frontend, erorile de rețea sau de validare sunt afișate într-un toast sau o notificare vizibilă. Pe backend, erorile sunt capturate într-un middleware centralizat care returnează un răspuns JSON formatat standard.

Pentru gestionarea memoriei și a resurselor, conexiunile la baza de date sunt gestionate printr-un pool de conexiuni oferit de driverul pg. Acest pool menține un număr limitat de conexiuni deschise și le reutilizează, evitând suprasolicitarea bazei de date. Setările implicite (10 conexiuni în pool) sunt suficiente pentru un proiect de dimensiunea acestuia.

3.2 Metode de dezvoltare

Abordarea generală

Proiectul folosește o metodologie de dezvoltare în cascadă adaptată, specifică proiectelor academice cu termene fixe. Aceasta înseamnă că etapele de analiză, proiectare, implementare și testare se desfășoară în ordine, dar există posibilitatea unor reveniri limitate între etape atunci când se descoperă probleme.

Această abordare a fost aleasă în locul metodologiilor agile (precum Scrum) deoarece proiectul are un singur dezvoltator și un termen de predare fix, ceea ce face ca planificarea predictibilă să

fie mai importantă decât flexibilitatea. De asemenea, cerințele sunt relativ bine definite încă de la început, ceea ce reduce necesitatea unor schimbări frecvente de direcție.

Metode formale utilizate

Pentru proiectarea arhitecturii se utilizează diagrame UML (Unified Modeling Language), în special diagrame de clasă pentru modelarea bazei de date și diagrame de secvență pentru modelarea interacțiunilor dintre componente. Aceste diagrame sunt incluse în anexele documentului de proiectare și în lucrarea de licență.

Pentru dezvoltarea software-ului se utilizează paradigma orientată pe obiecte acolo unde este cazul (de exemplu, pentru modelarea utilizatorilor și călătoriilor), combinată cu programarea funcțională pentru servicii precum algoritmul de optimizare. JavaScript permite ambele paradigme, iar alegerea între ele se face în funcție de ceea ce este mai natural pentru fiecare componentă.

Pentru gestionarea versiunilor se utilizează Git cu fluxul de lucru standard: un repository central pe GitHub, commit-uri frecvente cu mesaje descriptive, și un branch principal numit main care conține întotdeauna o versiune funcțională.

Metode luate în considerare și respinse

Metodologia Scrum a fost luată în considerare, dar a fost respinsă deoarece necesită o echipă (product owner, scrum master, echipă de dezvoltare) și sprint-uri regulate. Pentru un proiect individual, beneficiile Scrum sunt limitate în comparație cu efortul administrativ necesar.

Dezvoltarea bazată pe testare (Test-Driven Development) a fost luată în considerare parțial, dar a fost respinsă pentru UI deoarece testarea automată a interfeței cu utilizatorul este complexă și consumatoare de timp. Totuși, pentru algoritmul de optimizare se vor scrie teste unitare înainte de implementare.

Programarea în perechi (pair programming) nu este aplicabilă deoarece există un singur dezvoltator.

Contingențe și planuri alternative

Dacă API-ul OpenStreetMap devine indisponibil sau instabil pe durata dezvoltării, alternativa este utilizarea unei hărți statice (imagine fixă) pentru demonstrarea funcționalităților de bază. În acest caz, ruta nu va fi interactivă, dar algoritmul de optimizare va putea fi testat în continuare.

Dacă integrarea autentificării cu sesiuni prezintă probleme de securitate sau complexitate neașteptată, alternativa este utilizarea unui token simplu (JWT) sau chiar amânarea autentificării pentru o fază ulterioară, păstrând doar funcționalitatea de calcul al rutei pentru vizitatori.

Dacă PostgreSQL nu poate fi instalat sau configurat corect pe mediul de dezvoltare, alternativa este utilizarea SQLite ca bază de date embedded. SQLite nu necesită un server separat și este

mai ușor de configurat, dar oferă funcționalități limitate comparativ cu PostgreSQL. Această alternativă este considerată de ultimă instanță.

Dacă timpul de dezvoltare devine insuficient, planul de reducere a scopului este următorul: modulul de planificare cu algoritm de optimizare este obligatoriu; modulul de autentificare și istoric este important, dar poate fi simplificat; modulul de administrare și predicțiile de trafic sunt opționale și pot fi eliminate fără a afecta esența proiectului

3.3 Strategii de arhitectură

Abordarea generală

Proiectul folosește o metodologie de dezvoltare în cascadă adaptată, specifică proiectelor academice cu termene fixe. Aceasta înseamnă că etapele de analiză, proiectare, implementare și testare se desfășoară în ordine, dar există posibilitatea unor reveniri limitate între etape atunci când se descoperă probleme.

Această abordare a fost aleasă în locul metodologiilor agile (precum Scrum) deoarece proiectul are un singur dezvoltator și un termen de predare fix, ceea ce face ca planificarea predictibilă să fie mai importantă decât flexibilitatea. De asemenea, cerințele sunt relativ bine definite încă de la început, ceea ce reduce necesitatea unor schimbări frecvente de direcție.

Metode formale utilizate

Pentru proiectarea arhitecturii se utilizează diagrame UML (Unified Modeling Language), în special diagrame de clasă pentru modelarea bazei de date și diagrame de secvență pentru modelarea interacțiunilor dintre componente. Aceste diagrame sunt incluse în anexele documentului de proiectare și în lucrarea de licență.

Pentru dezvoltarea software-ului se utilizează paradigma orientată pe obiecte acolo unde este cazul (de exemplu, pentru modelarea utilizatorilor și călătoriilor), combinată cu programarea funcțională pentru servicii precum algoritmul de optimizare. JavaScript permite ambele paradigme, iar alegerea între ele se face în funcție de ceea ce este mai natural pentru fiecare componentă.

Pentru gestionarea versiunilor se utilizează Git cu fluxul de lucru standard: un repository central pe GitHub, commit-uri frecvente cu mesaje descriptive, și un branch principal numit main care conține întotdeauna o versiune funcțională.

Metode luate în considerare și respinse

Metodologia Scrum a fost luată în considerare, dar a fost respinsă deoarece necesită o echipă (product owner, scrum master, echipă de dezvoltare) și sprint-uri regulate. Pentru un proiect individual, beneficiile Scrum sunt limitate în comparație cu efortul administrativ necesar.

Dezvoltarea bazată pe testare (Test-Driven Development) a fost luată în considerare parțial, dar a fost respinsă pentru UI deoarece testarea automată a interfeței cu utilizatorul este complexă și consumatoare de timp. Totuși, pentru algoritmul de optimizare se vor scrie teste unitare înainte de implementare.

Programarea în perechi (pair programming) nu este aplicabilă deoarece există un singur dezvoltator.

Contingențe și planuri alternative

Dacă API-ul OpenStreetMap devine indisponibil sau instabil pe durata dezvoltării, alternativa este utilizarea unei hărți statice (imagine fixă) pentru demonstrarea funcționalităților de bază. În acest caz, ruta nu va fi interactivă, dar algoritmul de optimizare va putea fi testat în continuare.

Dacă integrarea autentificării cu sesiuni prezintă probleme de securitate sau complexitate neașteptată, alternativa este utilizarea unui token simplu (JWT) sau chiar amânarea autentificării pentru o fază ulterioară, păstrând doar funcționalitatea de calcul al rutei pentru vizitatori.

Dacă PostgreSQL nu poate fi instalat sau configurat corect pe mediul de dezvoltare, alternativa este utilizarea SQLite ca bază de date embedded. SQLite nu necesită un server separat și este mai ușor de configurat, dar oferă funcționalități limitate comparativ cu PostgreSQL. Această alternativă este considerată de ultimă instanță.

Dacă timpul de dezvoltare devine insuficient, planul de reducere a scopului este următorul: modulul de planificare cu algoritm de optimizare este obligatoriu; modulul de autentificare și istoric este important, dar poate fi simplificat; modulul de administrare și predicțiile de trafic sunt opționale și pot fi eliminate fără a afecta esența proiectului.

3.3 Strategii de arhitectură

Decizia 1: Alegerea limbajului și platformei backend

S-a decis utilizarea Node.js cu framework-ul Express pentru backend. Această decizie se bazează pe familiaritatea cu JavaScript, care este deja utilizat pe frontend, permițând astfel un schimb mai ușor de cunoștințe între cele două straturi. De asemenea, ecosistemul npm oferă o gamă largă de biblioteci utile, iar performanța asincronă a Node.js este potrivită pentru operațiile de I/O (acces la bază de date, apeluri API externe).

Alternativele luate în considerare au fost Python cu Django și Java cu Spring Boot. Django a fost respins deoarece necesită o curbă de învățare mai abruptă și este mai orientat către aplicații

monolitice mari. Spring Boot a fost respins deoarece Java este mai verbose și necesită mai mult cod pentru aceleași funcționalități, ceea ce ar fi consumat timp prețios de dezvoltare.

Decizia 2: Alegerea bazei de date

S-a decis utilizarea PostgreSQL ca sistem de gestiune a bazelor de date. PostgreSQL este open-source, robust, respectă standardele SQL și oferă suport pentru date geospațiale prin extensia PostGIS, care ar putea fi utilă pentru viitoare extensii ale proiectului.

Alternativa luată în considerare a fost MySQL. MySQL a fost respins deoarece nu oferă același nivel de conformitate cu standardele SQL și are suport mai limitat pentru interogări complexe și tipuri de date avansate. SQLite a fost luat în considerare ca alternativă ușoară, dar a fost respins deoarece nu suportă concurența bine și nu este potrivit pentru o arhitectură client-server.

Decizia 3: Arhitectura de comunicare

S-a decis utilizarea unei arhitecturi REST pentru API-ul backend. REST este simplu, bine înțeles și folosește metode HTTP standard (GET, POST, PUT, DELETE) ceea ce îl face ușor de implementat și de testat. Toate răspunsurile sunt în format JSON, care este ușor de procesat atât pe backend cât și pe frontend.

Alternativa luată în considerare a fost GraphQL. GraphQL a fost respins deoarece adaugă o complexitate semnificativă pentru beneficiile pe care le aduce (flexibilitate în interogări), iar aplicația nu are nevoie de această flexibilitate deoarece resursele sunt puține și bine definite.

Decizia 4: Mecanismul de autentificare

S-a decis utilizarea sesiunilor HTTP pentru autentificarea utilizatorilor. Sesiunile sunt gestionate de server printr-un middleware (express-session), iar browserul primește un cookie care identifică sesiunea. Această abordare este simplă de implementat și oferă un nivel bun de securitate pentru un proiect academic.

Alternativa luată în considerare a fost JWT (JSON Web Tokens). JWT a fost respins deoarece gestionează mai greu invalidarea token-urilor (de exemplu, la logout sau schimbarea parolei) și necesită o infrastructură mai complexă pentru refresh tokens. Pentru un proiect cu un număr mic de utilizatori, sesiunile sunt mai mult decât suficiente.

Decizia 5: Biblioteca pentru hartă

S-a decis utilizarea Leaflet.js împreună cu OpenStreetMap pentru afișarea hărții și a rutelor. Leaflet este ușoară, open-source și se integrează bine cu orice aplicație web. OpenStreetMap oferă hărți gratuite și fără restricții de utilizare pentru proiecte necomerciale.

Alternativa luată în considerare a fost Google Maps API. Google Maps a fost respins din cauza costurilor asociate după depășirea cotelor gratuite și a necesității de a furniza date de card

bancar pentru activarea contului. De asemenea, Google Maps impune restricții privind numărul de cereri gratuite, iar un proiect academic nu poate garanta că nu va depăși aceste cote.

Decizia 6: Planuri viitoare pentru extensii

Deși proiectul este o lucrare de licență cu ciclu de viață limitat, arhitectura a fost gândită pentru a permite eventuale extensii viitoare. Separarea clară pe straturi (frontend, backend, baza de date) permite înlocuirea oricărei componente fără a afecta celelalte. Adăugarea unui nou modul (de exemplu, integrarea cu un API real de trafic) necesită doar adăugarea unui nou serviciu și a unor noi endpoint-uri, fără a rescrie codul existent. Extensia PostGIS pentru PostgreSQL poate fi activată ulterior pentru a permite interogări geospațiale avansate (găsirea celor mai apropiate puncte de interes, calcularea distanțelor reale pe hartă).

Decizia 7: Detectarea și recuperarea erorilor

S-a implementat o strategie centralizată de gestionare a erorilor. Pe backend, există un middleware de erori care captează toate excepțiile neașteptate și returnează un răspuns JSON cu codul de stare HTTP corespunzător (400 pentru erori de client, 404 pentru resurse negăsite, 500 pentru erori interne). Pe frontend, există o funcție globală de procesare a răspunsurilor care afișează utilizatorului un mesaj prietenos în caz de eroare și loghează detaliile în consola dezvoltatorului.

Decizia 8: Gestionarea concurenței

Deoarece aplicația rulează pe un singur server și are un număr estimat scăzut de utilizatori simultani (sub 30), nu sunt necesare mecanisme avansate de concurență. Node.js utilizează un model asincron non-blocant care poate gestiona mii de conexiuni simultane fără probleme. Baza de date PostgreSQL gestionează automat concurența prin mecanismele sale native de tranzacții și izolare. Singura precauție luată este utilizarea unui pool de conexiuni la baza de date pentru a evita suprasolicitarea acesteia.

4. Arhitectura Sistemului și Proiectarea Arhitecturii

Această secțiune descrie arhitectura sistemului TripSmart Agency, modul în care aplicația interacționează cu alte aplicații și cum datele sunt transferate între componente. Scopul principal este de a oferi o înțelegere generală a modului și motivelor pentru care sistemul a fost descompus și cum părțile individuale lucrează împreună pentru a furniza funcționalitatea dorită.

Descompunerea sistemului în componente

Sistemul TripSmart Agency a fost descompus în patru subsisteme de nivel superior, fiecare având responsabilități clar definite. Această descompunere urmărește principiul separări preocupărilor (separation of concerns), un principiu fundamental în ingineria software care afirmă că un sistem este mai ușor de înțeles, dezvoltat și întreținut atunci când fiecare componentă are o responsabilitate bine definită și cât mai independentă posibil.

Primul subsistem este subsistemul de prezentare (frontend), a cărui responsabilitate este de a interacționa direct cu utilizatorul. Acesta afișează interfața grafică, colectează datele introduse de utilizator, trimite cereri către backend și afișează rezultatele primite. Subsistemul de prezentare nu conține logică de business și nu accesează direct baza de date.

Al doilea subsistem este subsistemul de logică de business (backend), a cărui responsabilitate este de a procesa cererile venite de la frontend, de a aplica regulile de business (în special algoritmul de optimizare a rutelor) și de a coordona accesul la date. Acesta expune un API REST prin care frontend-ul poate solicita operații.

Al treilea subsistem este subsistemul de stocare a datelor (baza de date), a cărui responsabilitate este de a persista datele aplicației (utilizatori, călătorii, date de trafic) și de a oferi mecanisme eficiente de interogare și actualizare.

Al patrulea subsistem este subsistemul extern de hartă (OpenStreetMap), a cărui responsabilitate este de a furniza date cartografice și funcționalități de geocodare. Acesta nu este dezvoltat în cadrul proiectului, ci este consumat ca serviciu extern.

Colaborarea între componente

Componentele de nivel superior colaborează după cum urmează pentru a realiza funcționalitatea de planificare a unei rute. Utilizatorul introduce datele în formularul din frontend. Frontend-ul validează sintactic datele și trimite o cerere HTTP POST către backend la endpoint-ul /api/trips/optimize, cu datele în format JSON. Backend-ul primește cererea, extrage parametrii și interoghează baza de date pentru a obține datele de trafic relevante. După ce primește răspunsul de la bază de date, backend-ul aplică algoritmul de optimizare multi-criterială și pregătește răspunsul. Backend-ul trimite răspunsul JSON către frontend, care la rândul său afișează ruta pe hartă (folosind API-ul extern OpenStreetMap) și rezultatele în panoul dedicat.

Justificarea descompunerii

Această descompunere în patru subsisteme a fost aleasă deoarece oferă mai multe avantaje. Separarea dintre prezentare și logică permite dezvoltarea independentă a frontend-ului și backend-ului, chiar și de către persoane diferite. Separarea dintre logică și date permite schimbarea bazei de date fără a afecta restul aplicației. Consumul unui serviciu extern de hartă reduce complexitatea proiectului, deoarece nu este necesară implementarea internă a funcționalităților cartografice.

Au fost luate în considerare și alte descompuneri, cum ar fi o arhitectură monolitică în care tot codul (frontend, backend, acces la date) se află în același proiect și se rulează pe același server. Această alternativă a fost respinsă deoarece face mai dificilă testarea independentă a componentelor și nu separă clar responsabilitățile. De asemenea, a fost luată în considerare o arhitectură de microservicii, în care fiecare modul (planificare, analiză, utilizator, admin) ar fi un serviciu separat. Această alternativă a fost respinsă deoarece adaugă o complexitate semnificativă (descoperirea serviciilor, comunicare între servicii, consistență distribuită) care nu este justificată de dimensiunea modestă a proiectului.

Tipare de proiectare utilizate

Proiectul utilizează mai multe tipare de proiectare recunoscute. Tiparul Model-Vedere-Controler (MVC) este utilizat pe backend pentru a separa logica de business (Modelele și Controlerele) de interfața cu utilizatorul (Vederile, care în cazul unui API REST sunt reprezentate de răspunsurile JSON). Tiparul Repository este utilizat pentru a abstractiza accesul la baza de date, astfel încât controlerele nu interacționează direct cu SQL ci cu funcții precum getUserByEmail sau saveTrip. Tiparul Middleware este utilizat în Express pentru a compune lanțul de procesare a cererilor, cu middleware-uri pentru autentificare, logging, parsare JSON și gestionarea erorilor. Tiparul Singleton este utilizat pentru pool-ul de conexiuni la baza de date, asigurând că există o singură instanță a pool-ului în întreaga aplicație.

Justificarea utilizării acestor tipare este că ele reprezintă soluții consacrate pentru probleme comune în dezvoltarea software. MVC este standard pentru aplicații web și face codul ușor de înțeles pentru alți dezvoltatori. Repository separă logica de acces la date de restul aplicației, facilitând testarea și eventuala schimbare a bazei de date. Middleware-ul oferă o modalitate elegantă de a adăuga funcționalități transversale (logging, autentificare) fără a modifica controlerele.

4.1 Vedere logică

Vederea logică a arhitecturii sistemului este reprezentată de cele patru module logice principale descrise în secțiunea anterioară: Modulul Planificare, Modulul Analiză Trafic, Modulul Utilizator și Modulul Administrare. Aceste module sunt organizate pe verticală în cadrul arhitecturii pe trei straturi (prezentare, logică, date).

Diagramele UML corespunzătoare vederii logice (diagramă de pachete, diagramă de clasă pentru baza de date) sunt incluse în Anexa B a acestui document.

4.2 Arhitectură hardware

Sistemul TripSmart Agency utilizează o arhitectură hardware centralizată, în care toate componentele rulează pe o singură mașină fizică. Aceasta este o alegere potrivită pentru un proiect de licență, deoarece simplifică dezvoltarea, testarea și demonstrarea aplicației.

Componentele hardware implicate sunt următoarele. Stația de dezvoltare (un laptop sau PC personal) găzduiește atât serverul de aplicație (Node.js), cât și serverul de baze de date (PostgreSQL), cât și browser-ul pentru accesul la interfața web. Această componentă necesită un procesor Intel Core i5 sau echivalent, minimum 8 GB RAM și 256 GB stocare SSD pentru performanțe optime. Nu sunt necesare servere separate pentru prezentare, aplicație sau date, deoarece toate rolurile sunt îndeplinite de aceeași mașină.

Nu sunt necesare dispozitive periferice specializate precum load balancers, acceleratori SSL, switch-uri dedicate sau firewall-uri hardware, deoarece aplicația rulează în mediu de dezvoltare local, fără expunere la internet. Singurele dispozitive periferice necesare sunt cele standard ale oricărui calculator: monitor, tastatură și mouse.

Conectivitatea între componente se face prin interfața de loopback (localhost), fără a traversa rețeaua fizică. Browser-ul accesează serverul Node.js la adresa http://localhost:3000. Serverul Node.js se conectează la baza de date PostgreSQL la adresa localhost:5432. Browser-ul se conectează la API-ul extern OpenStreetMap prin internet, la adresa https://api.openstreetmap.org.

Estimările resurselor hardware necesare sunt următoarele. Capacitatea procesorului necesară este de aproximativ 20% dintr-un nucleu pentru serverul Node.js în sarcină normală, și până la 50% în momentul calculului unei rute complexe. Memoria RAM necesară este de aproximativ 512 MB pentru Node.js, 256 MB pentru PostgreSQL și restul (minimum 4 GB) pentru sistemul de operare și browser. Stocarea online necesară este de aproximativ 100 MB pentru codul sursă al aplicației și 50-100 MB pentru baza de date în funcție de numărul de înregistrări. Stocarea auxiliară (backup-uri, documentație) necesită aproximativ 1 GB suplimentar.

4.3 Arhitectură software

Arhitectura software a sistemului TripSmart Agency este organizată pe trei straturi distincte, fiecare cu responsabilități bine delimitate și cu dependențe controlate.

Stratul de prezentare (frontend)

Acest strat este responsabil cu afișarea interfeței utilizator și cu gestionarea interacțiunilor directe cu utilizatorul. Componentele software incluse sunt: HTML5 pentru structura paginilor; CSS3 pentru stilizare și aspect vizual (cu suport pentru design responsive); JavaScript vanilla (ECMAScript 2020+) pentru logica de interacțiune, apeluri API și manipularea dinamică a DOM-ului; Leaflet.js versiunea 1.9.4 (licență BSD-2) pentru afișarea hărții interactive și a rutelor; Chart.js versiunea 4.4.0 (licență MIT) pentru generarea graficelor statistice în pagina de analiză.

Nu se utilizează framework-uri frontend precum React, Angular sau Vue deoarece adaugă complexitate și timp de învățare suplimentar, iar funcționalitățile necesare pot fi realizate eficient cu JavaScript vanilla.

Stratul de logică de business (backend)

Acest strat este responsabil cu procesarea cererilor, aplicarea regulilor de business și coordonarea accesului la date. Componentele software incluse sunt: Node.js versiunea 18.x (licență MIT) ca mediu de rulare JavaScript pe server; Express versiunea 4.18.x (licență MIT) ca framework web pentru gestionarea rutelor și middleware; bcrypt versiunea 5.1.x (licență MIT) pentru criptarea și verificarea parolelor; express-session versiunea 1.17.x (licență MIT) pentru gestionarea sesiunilor utilizatorilor; pg versiunea 8.11.x (licență MIT) ca driver pentru conectarea la PostgreSQL; cors versiunea 2.8.x (licență MIT) pentru a permite cereri cross-origin în timpul dezvoltării.

Backend-ul expune un API REST cu următoarele categorii de endpoint-uri. Endpoint-urile de autentificare (/api/auth/*) includ POST /api/auth/register pentru înregistrarea unui utilizator nou, POST /api/auth/login pentru autentificare și POST /api/auth/logout pentru încheierea sesiunii. Endpoint-urile de călătorii (/api/trips/*) includ POST /api/trips/optimize pentru calculul rutei optime, POST /api/trips/save pentru salvarea unei rute în istoric, GET /api/trips/history pentru listarea rutelor salvate și DELETE /api/trips/:id pentru ștergerea unei rute. Endpoint-urile de trafic (/api/traffic/*) includ GET /api/traffic/stats pentru obținerea statisticilor agregate de trafic. Endpoint-urile de administrare (/api/admin/*) includ GET /api/admin/users pentru listarea tuturor utilizatorilor (acces restricționat).

Stratul de date

Acest strat este responsabil cu stocarea și recuperarea persistentă a datelor. Componentele software incluse sunt: PostgreSQL versiunea 15.x (licență PostgreSQL) ca sistem de gestiune a bazelor de date; driverul pg care permite aplicației Node.js să se conecteze și să execute interogări SQL.

Tabelele din baza de date sunt: users pentru stocarea informațiilor utilizatorilor (id, name, email, password_hash, created_at, is_admin); trips pentru stocarea călătoriilor salvate (id, user_id, origin, destination, date, cost, duration, created_at); traffic_data pentru stocarea datelor istorice de trafic (id, zone, timestamp, congestion_level, average_speed, created_at).

Fluxul fizic al datelor

Fluxul datelor pentru operația principală (calcul rută optimă) este următorul. Frontend-ul colectează datele din formular (origine, destinație, dată, oră, criteriu) și le validează la nivel de client. Frontend-ul trimite o cerere HTTP POST la backend la adresa /api/trips/optimize, cu corpul cererii în format JSON conținând datele colectate. Backend-ul primește cererea, validează datele și autentificarea utilizatorului (dacă este cazul). Backend-ul interoghează baza de date pentru a prelua datele de trafic relevante pentru zona și intervalul orar specificate. Backend-ul aplică algoritmul de optimizare multi-criterială (scoring ponderat) pentru a calcula ruta optimă. Backend-ul construiește răspunsul JSON conținând ruta (listă de puncte geografice), timpul estimat, costul estimat și avertizările de trafic. Backend-ul trimite răspunsul HTTP către frontend. Frontend-ul afișează ruta pe hartă folosind Leaflet.js și OpenStreetMap, și afișează rezultatele textuale în panoul dedicat.

4.4 Arhitectura informațiilor

Sistemul TripSmart Agency stochează trei categorii principale de informații, fiecare cu caracteristici și cerințe diferite privind securitatea și confidențialitatea.

Categoria 1: Date despre utilizatori

Aceste informații includ numele complet al utilizatorului, adresa de email (folosită ca nume de utilizator la autentificare) și hash-ul parolei generate de bcrypt. Toate aceste date sunt considerate informații personale și, prin urmare, sunt clasificate ca având caracter sensibil. Data înregistrării (created_at) și privilegiile administrative (is_admin) sunt stocate împreună cu utilizatorul, dar nu sunt considerate sensibile în sine.

Datele sunt furnizate sistemului de către utilizatori prin intermediul formularului de înregistrare, unde acestea sunt introduse manual de către utilizator. Formatul datelor este electronic (text), iar transmiterea de la client la server se face prin HTTP/JSON. Parola este transmisă în clar doar în cererea de înregistrare sau autentificare, fiind criptată imediat în backend înainte de stocare.

Categoria 2: Date despre călătorii

Aceste informații includ originea, destinația, data, ora (implicită în timestamp), costul estimat și durata estimată a unei călătorii planificate sau salvate. Aceste date nu sunt considerate sensibile, deoarece nu conțin informații de identificare personală directă, dar sunt totuși asociate unui utilizator prin cheia străină user_id pentru a permite salvarea istoricului personalizat.

Datele sunt furnizate sistemului de către utilizatori prin intermediul formularului de planificare. Utilizatorul introduce manual originea și destinația (text) și selectează data și ora din interfețe dedicate. Costul și durata sunt calculate de sistem, nu sunt introduse de utilizator. Formatul datelor este electronic (text pentru locații, numere pentru cost și durată, dată și oră pentru timestamp).

Categoria 3: Date despre trafic

Aceste informații includ zona geografică (numele orașului sau al sectorului), timestamp-ul momentului înregistrării, nivelul de congestie (procent între 0 și 100) și viteza medie (km/h). Aceste date nu sunt considerate sensibile, sunt complet anonimizate și nu sunt asociate niciunui utilizator. Ele servesc exclusiv pentru analize statistice și pentru alimentarea algoritmului de optimizare.

Datele sunt furnizate sistemului de către un script intern de generare sintetică, nu de către utilizatori. În viitor, acestea ar putea fi înlocuite cu date reale de la un API extern. Formatul datelor este electronic, iar introducerea se face automat prin interogări SQL executate de backend la pornirea aplicației sau periodic.

4.5 Arhitectura de comunicații interne

Sistemul TripSmart Agency utilizează o arhitectură de comunicații simplă, centralizată în jurul serverului backend. Toate comunicațiile se fac prin protocoale standard și nu necesită echipamente de rețea specializate.

Comunicația dintre browser și backend

Această comunicație folosește protocolul HTTP pe portul 3000. Browser-ul acționează ca client, inițiind cereri către backend. Backend-ul acționează ca server, răspunzând cu coduri de stare HTTP și corp în format JSON. Metodele HTTP utilizate sunt GET pentru operații de citire (fără efecte secundare) și POST pentru operații de scriere sau calcul. Formatul mesajelor este JSON, cu antetul Content-Type: application/json. Pentru mediul de dezvoltare, nu se utilizează HTTPS din cauza complexității configurării certificatelor, dar în producție ar fi obligatoriu.

Comunicația dintre backend și baza de date

Această comunicație folosește protocolul TCP/IP pe portul 5432, care este portul implicit al PostgreSQL. Driverul pg gestionează conexiunea și folosește protocolul wire nativ al PostgreSQL pentru a trimite interogări SQL și a primi rezultate. Toate interogările sunt parametrizate pentru a preveni atacurile SQL injection. Conexiunea se face la adresa localhost deoarece baza de date rulează pe aceeași mașină.

Comunicația dintre browser și API-ul extern OpenStreetMap

Această comunicație folosește protocolul HTTPS pe portul 443, care este portul standard pentru trafic web securizat. Browser-ul face cereri directe către API-ul OpenStreetMap, fără a trece prin backend. Aceste cereri sunt de două tipuri: cereri de tile-uri hartă (imagini) pentru afișarea stratului cartografic de bază, și cereri de geocodare (transformarea numelor de locații în coordonate). API-ul OpenStreetMap nu necesită autentificare pentru utilizare de bază, dar impune rate limiting pentru a preveni abuzurile.

Estimări ale resurselor de rețea

Pentru mediul de dezvoltare pe localhost, toate comunicațiile sunt interne mașinii, deci lățimea de bandă necesară este neglijabilă (practic limitată de viteza magistralei sistemului). Pentru comunicația cu API-ul extern OpenStreetMap, lățimea de bandă necesară este de aproximativ 100-500 KB per încărcare de pagină (pentru tile-urile hărții). Aceasta este bine încadrată în orice conexiune standard de internet (minimum 10 Mbps). Nu sunt necesare capacități speciale de rețea LAN sau WAN.

4.6 Diagrama de arhitectură a sistemului

Diagrama de arhitectură integrată a sistemului TripSmart Agency prezintă toate componentele hardware și software și modul în care acestea interacționează.

La nivelul superior se află utilizatorul, care poate fi vizitator (neautentificat), utilizator autentificat sau administrator. Utilizatorul interacționează cu sistemul printr-un browser web (Chrome, Firefox sau Edge) care rulează pe aceeași mașină fizică.

Browser-ul găzduiește stratul de prezentare al aplicației. Acesta include șase pagini principale: Acasă (prezentare aplicație), Planificare (motorul principal de calcul rute), Analiză (statistici trafic), Istoric (rute salvate), Cont (autentificare și profil) și Admin (panou administrare, vizibil doar pentru administratori). Componentele frontend includ formularul de rută cu validare, harta interactivă realizată cu Leaflet.js care consumă API-ul OpenStreetMap, și graficele realizate cu Chart.js.

Browser-ul comunică cu backend-ul Node.js prin HTTP pe portul 3000, folosind formatul JSON pentru date. Backend-ul este organizat pe mai multe straturi interne. Stratul Routes definește endpoint-urile API grupate pe domenii: auth (autentificare), trips (călătorii), traffic (trafic) și admin (administrare). Stratul Controllers procesează cererile și orchestrează răspunsurile. Stratul Services conține logica de business complexă, în special OptimizationService care implementează algoritmul de scoring ponderat. Stratul Models abstractizează accesul la baza de date. Stratul Middleware include funcții pentru autentificare, logging, CORS și gestionarea erorilor.

Backend-ul comunică cu baza de date PostgreSQL prin TCP pe portul 5432, folosind SQL interogări parametrizate. Baza de date conține trei tabele: users (id, name, email, password_hash, created_at, is_admin), trips (id, user_id, origin, destination, date, cost, duration, created_at) și traffic_data (id, zone, timestamp, congestion_level, average_speed, created_at).

Browser-ul comunică direct cu API-ul extern OpenStreetMap prin HTTPS pe portul 443, pentru a obține tile-urile hărții și funcționalitățile de geocodare. Această comunicație nu trece prin backend pentru a reduce încărcarea serverului și a beneficia de cache-ul browser-ului.

Toate componentele rulează pe aceeași mașină fizică (laptop-ul sau PC-ul de dezvoltare), ceea ce face ca toate comunicațiile să fie interne (localhost) cu excepția apelurilor către OpenStreetMap care traversează internetul.

5. Proiectarea sistemului

5.1 Proiectarea bazei de date

Sistemul TripSmart Agency utilizează PostgreSQL ca sistem de gestionare a bazelor de date pentru stocarea persistentă a datelor. Nu sunt utilizate fișiere non-DBMS pentru stocarea datelor, toate informațiile fiind gestionate prin intermediul bazei de date relationale.

5.1.1 Obiecte de date și structuri de date rezultante

Sistemul TripSmart Agency utilizează PostgreSQL ca sistem de gestionare a bazelor de date pentru stocarea persistentă a datelor. Nu sunt utilizate fișiere non-DBMS pentru stocarea datelor, toate informațiile fiind gestionate prin intermediul bazei de date relationale.

5.1.2 5.1.1 Obiecte de date și structuri de date rezultante

Dicționar de date - Tabela users

Tabela users stochează informațiile despre utilizatorii platformei.

Câmpul id este de tip SERIAL, are lungimea de 4 bytes, este cheia primară a tabelei și este obligatoriu. Sursa acestui câmp este baza de date care generează automat valori secvențiale. Regula de validare este că valoarea trebuie să fie unică și pozitivă. Operațiile CRUD permise sunt crearea la înregistrarea unui utilizator nou, citirea pentru autentificare și afișarea profilului, actualizarea niciodată (id-ul nu se modifică), ștergerea la dezactivarea contului. Stocarea se face în baza de date, iar ieșirea este utilizată în răspunsurile API pentru identificarea utilizatorului. Aliasul utilizat este user_id în tabelele care fac referire la această tabelă. Descrierea este "identificator unic al utilizatorului".

Câmpul name este de tip VARCHAR cu lungimea maximă de 100 de caractere și este obligatoriu. Sursa este formularul de înregistrare completat de utilizator. Regula de validare impune ca șirul să nu fie gol, să aibă între 2 și 100 de caractere și să conțină doar litere, spații și cratime. Operațiile CRUD sunt crearea la înregistrare, citirea pentru afișarea profilului, actualizarea la editarea profilului, ștergerea la ștergerea contului. Descrierea este "numele complet al utilizatorului".

Câmpul email este de tip VARCHAR cu lungimea maximă de 255 de caractere, este unic și obligatoriu. Sursa este formularul de înregistrare. Regula de validare impune respectarea formatului standard de email (local-part@domain), lungimea totală sub 255 de caractere și unicitatea în tabelă. Operațiile CRUD sunt crearea la înregistrare, citirea la autentificare și afișarea profilului, actualizarea la modificarea emailului, ștergerea la ștergerea contului. Descrierea este "adresa de email folosită pentru autentificare".

Câmpul password_hash este de tip VARCHAR cu lungimea maximă de 255 de caractere și este obligatoriu. Sursa este backend-ul care criptează parola primită de la utilizator folosind bcrypt. Regula de validare este că hash-ul trebuie să fie generat de bcrypt cu un factor de cost de 10. Operațiile CRUD sunt crearea la înregistrare, citirea la autentificare pentru verificarea parolei, actualizarea la schimbarea parolei, ștergerea la ștergerea contului. Descrierea este "hash-ul parolei utilizatorului generat de bcrypt".

Câmpul created_at este de tip TIMESTAMP cu valoarea implicită CURRENT_TIMESTAMP. Sursa este baza de date care adaugă automat timestamp-ul curent. Regula de validare este că valoarea trebuie să fie un timestamp valid. Operațiile CRUD sunt crearea automată la inserarea unui nou utilizator, citirea pentru afișarea datei înregistrării, nu se actualizează niciodată, se șterge odată cu utilizatorul. Descrierea este "data și ora înregistrării contului".

Câmpul is_admin este de tip BOOLEAN cu valoarea implicită FALSE. Sursa este backend-ul care poate seta această valoare pentru utilizatorii privilegiați. Regula de validare este că valoarea poate fi TRUE sau FALSE. Operațiile CRUD sunt crearea cu valoarea implicită FALSE, citirea pentru verificarea privilegiilor, actualizarea de către administrator, ștergerea odată cu utilizatorul. Descrierea este "flag care indică dacă utilizatorul are privilegii de administrator".

Dicționar de date - Tabela trips

Tabela trips stochează călătoriile salvate de utilizatori în istoricul personal.

Câmpul id este de tip SERIAL, cheie primară, obligatoriu. Sursa este baza de date. Regula de validare este unicitatea și valoare pozitivă. Operațiile CRUD sunt crearea la salvarea unei rute, citirea pentru afișarea istoricului, actualizarea niciodată, ștergerea la cererea utilizatorului. Descrierea este "identificator unic al călătoriei".

Câmpul user_id este de tip INTEGER, cheie străină către users.id, obligatoriu. Sursa este backend-ul care preia id-ul utilizatorului autentificat din sesiune. Regula de validare este că valoarea trebuie să existe în tabela users. Operațiile CRUD sunt crearea la salvarea rutei, citirea pentru filtrarea istoricului pe utilizator, actualizarea niciodată, ștergerea în cascadă la ștergerea utilizatorului. Descrierea este "identificatorul utilizatorului care a salvat călătoria".

Câmpul origin este de tip VARCHAR cu lungimea maximă de 200 de caractere, obligatoriu. Sursa este formularul de planificare completat de utilizator. Regula de validare impune ca șirul să nu fie gol și să reprezinte o locație geografică validă. Operațiile CRUD sunt crearea la salvarea rutei, citirea pentru afișarea istoricului, actualizarea niciodată, ștergerea la ștergerea înregistrării. Descrierea este "punctul de plecare al călătoriei".

Câmpul destination este de tip VARCHAR cu lungimea maximă de 200 de caractere, obligatoriu. Sursa este formularul de planificare. Regula de validare aceeași ca la origin. Descrierea este "destinația călătoriei".

Câmpul date este de tip DATE, obligatoriu. Sursa este selectorul de dată din formular. Regula de validare impune ca data să fie în format valid (YYYY-MM-DD) și în mod normal în viitor (deși nu este strict). Descrierea este "data călătoriei planificate".

Câmpul cost este de tip DECIMAL(10,2), opțional (poate fi NULL pentru rutele care nu au cost calculat). Sursa este algoritmul de optimizare care estimează costul. Regula de validare impune valoarea mai mare sau egală cu 0. Descrierea este "costul estimat al călătoriei în lei".

Câmpul duration este de tip INTEGER, opțional. Sursa este algoritmul de optimizare. Regula de validare impune valoarea mai mare sau egală cu 0. Descrierea este "durata estimată a călătoriei în minute".

Câmpul created_at este de tip TIMESTAMP cu valoarea implicită CURRENT_TIMESTAMP. Sursa este baza de date. Descrierea este "data și ora salvării călătoriei în istoric".

Dicționar de date - Tabela traffic_data

Tabela traffic_data stochează datele istorice de trafic pentru analize statistice.

Câmpul id este de tip SERIAL, cheie primară, obligatoriu. Descrierea este "identificator unic al înregistrării de trafic".

Câmpul zone este de tip VARCHAR cu lungimea maximă de 100 de caractere, obligatoriu. Sursa este scriptul de generare a datelor simulare. Regula de validare impune ca șirul să nu fie gol. Descrierea este "zona geografică pentru care se înregistrează traficul (ex: Centru, Nord, Sud)".

Câmpul timestamp este de tip TIMESTAMP, obligatoriu. Sursa este scriptul de generare. Descrierea este "momentul la care au fost înregistrate datele de trafic".

Câmpul congestion_level este de tip INTEGER, opțional. Sursa este scriptul de generare. Regula de validare impune valoarea între 0 și 100 inclusiv. Descrierea este "nivelul de congestie a traficului în procente (0=liber, 100=blocat)".

Câmpul average_speed este de tip INTEGER, opțional. Sursa este scriptul de generare. Regula de validare impune valoarea mai mare sau egală cu 0. Descrierea este "viteza medie a vehiculelor în kilometri pe oră".

Câmpul created_at este de tip TIMESTAMP cu valoarea implicită CURRENT_TIMESTAMP. Sursa este baza de date. Descrierea este "data și ora înregistrării în sistem".

5.1.3 Fișiere și baze de date

Structura fizică a bazei de date

Numele bazei de date este tripsmart_db. Aceasta este găzduită pe același server local (localhost) ca și aplicația backend. Locația fizică a fișierelor bazei de date depinde de instalarea PostgreSQL, de obicei în /var/lib/postgresql/ pe Linux sau în C:\Program Files\PostgreSQL\data pe Windows.

Tabela users

Această tabelă are următoarea structură SQL:

```
CREATE TABLE users (
id SERIAL PRIMARY KEY,
name VARCHAR(100) NOT NULL,
email VARCHAR(255) UNIQUE NOT NULL,
password_hash VARCHAR(255) NOT NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
is_admin BOOLEAN DEFAULT FALSE
);
```

Cheia primară este id. Indexul unic idx_users_email este creat pe coloana email pentru căutări rapide la autentificare.

Tabela trips

Această tabelă are următoarea structură SQL:

```
CREATE TABLE trips (
id SERIAL PRIMARY KEY,
user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,
origin VARCHAR(200) NOT NULL,
destination VARCHAR(200) NOT NULL,
date DATE NOT NULL,
cost DECIMAL(10,2) CHECK (cost >= 0),
duration INTEGER CHECK (duration >= 0),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Cheia primară este id. Cheia străină user_id face referire la users(id) cu ștergere în cascadă. Indexul idx_trips_user_id este creat pe coloana user_id pentru interogări eficiente ale istoricului unui utilizator.

Tabela traffic_data

Această tabelă are următoarea structură SQL:

```
CREATE TABLE traffic_data (
id SERIAL PRIMARY KEY,
zone VARCHAR(100) NOT NULL,
```

```
timestamp TIMESTAMP NOT NULL,
congestion_level INTEGER CHECK (congestion_level BETWEEN 0 AND 100),
average_speed INTEGER CHECK (average_speed >= 0),
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Cheia primară este id. Indexul idx_traffic_zone_time este creat pe coloanele (zone, timestamp) pentru interogări rapide după zonă și interval temporal.

Estimarea dimensiunii și volumului datelor

Pentru tabela users, dimensiunea estimată este de aproximativ 100 de înregistrări pentru faza de testare și demonstrație. Fiecare înregistrare ocupă aproximativ 500 de bytes, rezultând o dimensiune totală de aproximativ 50 KB.

Pentru tabela trips, se estimează aproximativ 500 de înregistrări (5 călătorii per utilizator pentru 100 de utilizatori). Fiecare înregistrare ocupă aproximativ 200 de bytes, rezultând o dimensiune totală de aproximativ 100 KB.

Pentru tabela traffic_data, se estimează aproximativ 10.000 de înregistrări pentru acoperirea a 10 zone pe parcursul a 30 de zile cu înregistrări la fiecare oră. Fiecare înregistrare ocupă aproximativ 100 de bytes, rezultând o dimensiune totală de aproximativ 1 MB.

Frecvența de actualizare

Pentru tabela users, operațiile de creare (înregistrare) sunt rare, estimate la 1-2 pe zi în faza de testare. Operațiile de citire (autentificare, afișare profil) sunt frecvente, estimate la 100-200 pe zi. Operațiile de actualizare (editare profil) sunt rare.

Pentru tabela trips, operațiile de creare (salvare rută) sunt estimate la 10-20 pe zi. Operațiile de citire (afișare istoric) sunt estimate la 30-50 pe zi. Operațiile de ștergere sunt rare.

Pentru tabela traffic_data, datele sunt încărcate în bloc la inițializarea sistemului și nu sunt actualizate frecvent în timpul demonstrației. Operațiile de citire sunt frecvente în timpul calculului rutelor și al afișării graficelor.

Specificații de backup și recuperare

Pentru backup-ul bazei de date se folosește comanda pg_dump: pg_dump -U postgres -d tripsmart_db > tripsmart_db_backup.sql. Backup-urile sunt realizată periodic (săptămânal) și stocate într-un director separat de baza de date principală. Pentru recuperare, se folosește comanda psql: psql -U postgres -d tripsmart_db < tripsmart_db_backup.sql.

5.1.3.1 Baze de date

Sistemul TripSmart Agency utilizează PostgreSQL ca sistem de gestionare a bazelor de date. Mai jos este prezentată proiectarea detaliată a fișierelor DBMS.

5.1.3.2 Fișiere non-DBMS

Sistemul TripSmart Agency utilizează și o serie de fișiere non-DBMS pentru stocarea codului sursă, a resurselor statice și a configurațiilor. Acestea nu sunt gestionate de sistemul de baze de date, ci de sistemul de fișiere al sistemului de operare.

Fișierele statice ale frontend-ului

Aceste fișiere includ paginile HTML, fișierele CSS, scripturile JavaScript și imaginile. Ele sunt utilizate exclusiv pentru intrare (citite de browser pentru a afișa interfața utilizatorului). Nu sunt fișiere temporare.

Fișierele HTML (index.html, planificare.html, analiza.html, istoric.html, cont.html, admin.html) conțin structura paginilor web. Ele sunt citite de browser la navigarea utilizatorului și nu sunt scrise niciodată de aplicație. Dimensiunea fiecărui fișier HTML este de aproximativ 5-15 KB.

Fișierele CSS (style.css) conțin regulile de stilizare. Sunt citite de browser și nu sunt modificate de aplicație. Dimensiunea este de aproximativ 20-30 KB.Fișierele JavaScript (map.js, chart.js, auth.js, api.js) conțin codul logic al frontend-ului. Sunt citite de browser și executate. Dimensiunea totală este de aproximativ 50-100 KB.

Imaginile (logo, iconițe, imagini de fundal) sunt stocate în folderul assets/images. Dimensiunea totală este de aproximativ 1-2 MB.

Modulul care citește aceste fișiere este browser-ul utilizatorului. Serverul Node.js le servește ca fișiere statice prin intermediul middleware-ului express.static. Nu există un modul care să scrie aceste fișiere în timpul execuției aplicației; ele sunt create o singură dată în faza de dezvoltare și modificate manual când este nevoie.

Structura acestor fișiere este cea standard pentru fișiere text: pentru HTML și CSS sunt fișiere text cu cod structurat, pentru JavaScript sunt fișiere text cu cod executabil, pentru imagini sunt fișiere binare în format PNG, JPEG sau SVG. Nu există structuri de înregistrare sau chei de indexare în sensul bazei de date.

Fișierele de configurare ale backend-ului

Aceste fișiere includ fișiere de configurare pentru aplicația Node.js și pentru conexiunea la baza de date. Principalul fișier de configurare este config/database.js, care conține parametrii de conexiune la PostgreSQL (host, port, nume baza de date, nume utilizator, parolă). Acest fișier este citit de modulul de configurare la pornirea serverului și nu este scris în timpul execuției. Este un fișier text JavaScript (JavaScript Object Notation sau cod JavaScript). Dimensiunea este de aproximativ 1-2 KB.

Fișierul .env (dacă este utilizat) stochează variabilele de mediu, inclusiv parolele și cheile de configurare sensibile. Acest fișier nu este inclus în repository-ul Git pentru a preveni expunerea informațiilor sensibile. Este citit la pornirea aplicației și nu este scris în timpul execuției.

Fișierele de log

Aplicația backend poate genera fișiere de log pentru înregistrarea evenimentelor și erorilor. Aceste fișiere sunt utilizate atât pentru intrare (citite de dezvoltator pentru depanare) cât și pentru ieșire (scrise de aplicație). Sunt fișiere temporare în sensul că pot fi șterse periodic fără a afecta funcționarea aplicației.

Fișierul de log (de exemplu, logs/app.log) conține linii de text, fiecare linie reprezentând o înregistrare de log cu timestamp, nivel (info, warn, error) și mesaj. Structura înregistrării este: [timestamp] [nivel] [mesaj]. De exemplu: [2026-04-26T10:30:00.000Z] [INFO] Server started on port 3000.

Dimensiunea fișierului de log depinde de durata de rulare. Pentru o sesiune de demonstrație de câteva ore, dimensiunea este de ordinul sutelor de KB. Frecvența de actualizare este de câteva zeci de linii pe oră în funcție de numărul de cereri.

Modulul care scrie în acest fișier este sistemul de logging al aplicației (de exemplu, middleware-ul de logging sau console.log redirecționat). Modulul care citește fișierul este dezvoltatorul, folosind un editor de text sau comanda tail în terminal.

Specificații de backup și recuperare pentru fișiere non-DBMS

Toate fișierele non-DBMS (cod sursă, resurse statice, configurații) sunt gestionate prin sistemul de versionare Git și sunt stocate în repository-ul GitHub. Backup-ul este asigurat de platforma GitHub, iar recuperarea se face prin clonarea repository-ului.

Pentru fișierele care nu sunt în repository (cum ar fi fișierul .env cu parole), acestea trebuie salvate separat, de exemplu într-un document protejat cu parolă sau într-un director de backup local. În caz de pierdere, fișierul .env poate fi recreat deoarece structura sa este cunoscută (doar valorile sunt sensibile).

Pentru fișierele de log, nu este necesar backup deoarece ele nu conțin date critice. În caz de pierdere, aplicația va crea fișiere noi automat

5.2 Conversii de date

Nu sunt necesare conversii de date de la sisteme legacy, deoarece platforma TripSmart Agency este dezvoltată de la zero și nu trebuie să importe date din sisteme existente. Toate datele sunt create direct de utilizatori prin intermediul formularelor aplicației sau sunt generate sintetic de scripturile de inițializare

5.3 Interfețe utilizator

Clase de utilizatori

Platforma TripSmart Agency definește trei clase de utilizatori, fiecare cu caracteristici și permisiuni diferite.

Prima clasă este Vizitatorul (neautentificat). Acești utilizatori nu au un cont pe platformă sau nu sunt autentificați. Responsabilitățile lor includ vizualizarea paginii Acasă, utilizarea motorului de planificare pentru calcularea rutelor (fără a le putea salva) și vizualizarea paginii de Analiză Trafic. Nivelul de competență tehnică este variabil, de la începător la avansat. Modul de interacțiune cu sistemul este prin intermediul browserului, folosind formulare și elemente grafice interactive. Numărul total estimat de vizitatori este nelimitat în teorie, dar pentru demonstrație se estimează 50-100. Numărul maxim de vizitatori simultani este estimat la 30. Nu există utilizatori externi în afara sistemului.

A doua clasă este Utilizatorul autentificat. Acești utilizatori și-au creat un cont și sunt autentificați în sistem. Responsabilitățile includ toate cele ale vizitatorilor, plus salvarea rutelor în istoric, vizualizarea istoricului personal, gestionarea profilului propriu și salvarea rutelor favorite. Nivelul de competență tehnică este similar cu vizitatorii. Numărul total estimat este de 50-100 de conturi create. Numărul maxim de utilizatori autentificați simultani este estimat la 20.

A treia clasă este Administratorul. Acești utilizatori au același cont ca un utilizator autentificat, dar cu flag-ul is_admin setat la TRUE. Responsabilitățile includ toate cele ale utilizatorilor autentificați, plus accesul la panoul de administrare pentru gestionarea utilizatorilor (listare, editare, ștergere) și vizualizarea statisticilor globale ale platformei. Nivelul de competență tehnică poate fi mai ridicat. Numărul total estimat este de 1-3 administratori (studentul autor și eventual coordonatorul pentru testare). Numărul maxim simultan este 1

5.3.1 Intrări

Mijloace de intrare

Toate intrările în sistem sunt realizate de utilizatori prin intermediul interfeței grafice web, folosind mouse-ul și tastatura. Principalele ecrane de introducere a datelor sunt formularul de înregistrare (câmpuri: nume, email, parolă), formularul de autentificare (câmpuri: email, parolă), formularul de planificare (câmpuri: origine, destinație, dată, oră, criteriu de optimizare) și formularul de editare a profilului (câmpuri: nume, email, parolă nouă).

Ecranul de înregistrare

Ecranul de înregistrare conține trei câmpuri de intrare. Câmpul nume este de tip text, obligatoriu, cu lungimea între 2 și 100 de caractere, acceptă litere, spații și cratime. Validarea se face în timp real: dacă câmpul este gol sau prea scurt, se afișează mesajul "Numele trebuie să

aibă între 2 și 100 de caractere". Câmpul email este de tip email, obligatoriu, cu format standard. Validarea verifică prezența caracterului @ și a unui domeniu valid. Mesajul de eroare este "Introduceți o adresă de email validă". Câmpul parolă este de tip parolă, obligatoriu, cu lungimea minimă de 6 caractere. Mesajul de eroare este "Parola trebuie să aibă cel puțin 6 caractere". Există un câmp suplimentar "confirmă parola" care verifică dacă cele două parole coincid. Butonul de trimitere este activat doar după ce toate validările sunt trecute.

Ecranul de autentificare

Ecranul de autentificare conține două câmpuri. Câmpul email este de tip email, obligatoriu. Câmpul parolă este de tip parolă, obligatoriu. După trimitere, dacă credentialele sunt incorecte, se afișează mesajul "Email sau parolă incorecte".

Ecranul de planificare

Ecranul de planificare conține următoarele câmpuri. Câmpul origine este de tip text, obligatoriu, lungime între 2 și 200 de caractere. Câmpul destinație este de tip text, obligatoriu, aceleași reguli. Câmpul dată este de tip date, obligatoriu, format YYYY-MM-DD. Validarea verifică dacă data este în format valid (nu neapărat în viitor). Câmpul oră este de tip time, obligatoriu, format HH:MM. Grupul de criterii de optimizare este de tip radio button, cu opțiunile: "Timp minim", "Cost minim", "Trafic redus", "Distanță minimă", "Eco-friendly". Una dintre opțiuni este selectată implicit (de obicei "Timp minim"). Toate câmpurile sunt validate pe client înainte de trimiterea cererii către backend. Dacă vreun câmp obligatoriu lipsește, butonul "Calculează ruta" este dezactivat

5.3.2 Ieșiri

Ecranul de planificare - rezultate

După trimiterea formularului și calcularea rutei, sistemul afișează următoarele ieșiri. Pe hartă se afișează ruta calculată sub forma unei linii colorate (de obicei albastru) care conectează originea și destinația, cu puncte marcate pentru start și sfârșit. În panoul de rezultate se afișează text: timpul estimat (în ore și minute), costul estimat (în lei), distanța (în kilometri) și nivelul de trafic (scăzut, mediu, ridicat). Dacă utilizatorul este autentificat, apare și un buton "Salvează în istoric". Scopul acestor ieșiri este de a oferi utilizatorului informațiile necesare pentru a decide dacă ruta calculată este potrivită. Nu există restricții de acces pentru aceste ieșiri.

Ecranul de istoric

Ecranul de istoric afișează un tabel cu coloanele: origine, destinație, dată, cost, durată, acțiuni. Fiecare rând reprezintă o călătorie salvată de utilizatorul autentificat. Coloana acțiuni conține butoane "Vezi pe hartă" (reîncarcă ruta în planificator) și "Șterge" (elimină din istoric). Scopul ieșirii este de a permite utilizatorului să își revadă călătoriile anterioare și să le reutilizeze. Accesul este restricționat la utilizatorul autentificat proprietar al datelor.

Ecranul de analiză trafic

Ecranul de analiză trafic afișează un set de grafice generate cu Chart.js. Graficul de congestie pe intervale orare este un grafic de tip linie care arată cum variază congestia pe parcursul zilei (medie pe toate zonele). Graficul vitezei medii pe zone este un grafic de tip bară care compară viteza medie în diferite zone ale orașului. Scopul este de a oferi o perspectivă asupra tiparelor de trafic, utilă pentru înțelegerea modului în care funcționează algoritmul de optimizare. Accesul este public (disponibil și vizitatorilor).

5.4 Proiectarea interfețelor cu utilizatorul

Pagina Acasă

Pagina Acasă este pagina de prezentare a aplicației. Structura sa include un antet cu logo-ul TripSmart Agency și meniul de navigare (Acasă, Planificare, Analiză, Cont). Secțiunea principală conține un titlu atractiv ("Planifică-ți călătoria inteligent cu TripSmart"), o scurtă descriere a aplicației și un butou de acțiune ("Începe acum") care redirecționează către pagina de planificare. Urmează o secțiune "Cum funcționează" care prezintă cei trei pași într-un format vizual: pasul 1 "Introdu datele călătoriei tale", pasul 2 "Selectează criteriile de optimizare preferate", pasul 3 "Primește ruta optimă pe hartă". O secțiune "Tehnologii utilizate" listează tehnologiile folosite (Node.js, PostgreSQL, Leaflet, Chart.js) cu iconițe reprezentative. La final se află un subsol cu informații de contact și anul curent.

Pagina Planificare

Pagina Planificare este motorul principal al aplicației. Este organizată pe două coloane. Coloana din stânga (aproximativ 30% din lățime) conține formularul de intrare. Coloana din dreapta (aproximativ 70% din lățime) conține harta interactivă realizată cu Leaflet.js, cu capacitatea de a face zoom și pan. Formularul conține câmpurile pentru origine și destinație, un câmp pentru dată (calendar pop-up), un câmp pentru oră (selector de timp), și un grup de butoane radio pentru selectarea criteriului de optimizare. Sub formular se află butonul "Calculează ruta" și un panou de rezultate care devine vizibil doar după calcul, afișând timpul estimat, costul estimat, distanța și nivelul de trafic.

Pagina Analiză

Pagina Analiză conține un titlu și două grafice principale. Graficul de congestie pe intervale orare este plasat în partea de sus, ocupând toată lățimea. Graficul vitezei medii pe zone este plasat dedesubt. Sub grafice, există o scurtă explicație a semnificației datelor și o mențiune că datele de trafic sunt simulate în scop demonstrativ.

Pagina Istoric

Pagina Istoric conține un tabel cu rutele salvate de utilizatorul autentificat. Coloanele tabelului sunt: Origine, Destinație, Dată, Cost, Durată, Acțiuni. Fiecare rând are un buton "Șterge" care elimină ruta din istoric după o confirmare. Deasupra tabelului există un buton "Șterge tot istoricul" care elimină toate rutele utilizatorului. Dacă utilizatorul nu are rute salvate, se afișează un mesaj "Nu aveți rute salvate. Planificați o călătorie și salvați-o pentru a apărea aici."

Pagina Cont

Pagina Cont are două stări. Dacă utilizatorul nu este autentificat, afișează două formulare unul lângă altul: formularul de autentificare (email, parolă, buton "Autentificare") și formularul de înregistrare (nume, email, parolă, confirmare parolă, buton "Înregistrare"). Dacă utilizatorul este autentificat, afișează formularul de editare a profilului cu câmpurile nume, email, parolă nouă, confirmare parolă nouă, plus un buton "Salvează modificările" și un buton "Deconectare".

Pagina Admin

Pagina Admin este accesibilă doar utilizatorilor cu is_admin = TRUE. Conține un tabel cu toți utilizatorii înregistrați, afișând pentru fiecare numele, emailul, data înregistrării și un buton "Șterge". Deasupra tabelului se află o secțiune de statistici globale care afișează numărul total de utilizatori, numărul total de călătorii și un mic grafic cu evoluția numărului de utilizatori în timp. Dacă un utilizator neautorizat încearcă să acceseze această pagină, este redirecționat către pagina Acasă cu un mesaj de eroare.

6. Scenarii de utilizare

Această secţiune descrie funcţionalitatea generală a sistemului TripSmart Agency din perspectiva utilizatorilor, furnizând fluxuri operaţionale pas cu pas pentru toate clasele de utilizatori identificate. Scenariile acoperă atât cazurile de succes, cât și situațiile de eroare.

6.1 Scenariul 1: Vizitatorul calculează o rută fără a fi autentificat

Clasă utilizator: Vizitator (neautentificat)
Scop: Calcularea unei rute optime între două locații
Mod operațional: Normal

Descrierea scenariului: Un vizitator accesează platforma și dorește să calculeze cea mai bună rută de la București la Brașov, prioritizând timpul minim.

Fluxul pas cu pas:

Pasul 1: Utilizatorul deschide browserul și navighează la adresa aplicației (http://localhost:3000). Sistemul încarcă pagina Acasă și afișează meniul de navigare și o prezentare generală a aplicației.

Pasul 2: Utilizatorul dă click pe link-ul "Planificare" din meniul de navigare. Sistemul încarcă pagina de planificare, care afișează formularul gol și harta interactivă centrată pe România.

Pasul 3: Utilizatorul completează câmpul "Plecare" cu textul "București". Sistemul validează că câmpul nu este gol și afișează sugestii de completare (dacă este implementat).

Pasul 4: Utilizatorul completează câmpul "Destinație" cu textul "Brașov". Sistemul validează câmpul și pregătește datele pentru trimitere.

Pasul 5: Utilizatorul selectează data "25-05-2026" folosind selectorul de dată. Sistemul afișează calendarul și, după selectare, completează câmpul cu data în formatul corect.

Pasul 6: Utilizatorul selectează ora "10:00" folosind selectorul de oră. Sistemul afișează câmpul cu ora selectată.

Pasul 7: Utilizatorul selectează criteriul de optimizare "Timp minim" din grupul de butoane radio. Sistemul reține această selecție și o evidențiază vizual.

Pasul 8: Utilizatorul apasă butonul "Calculează ruta". Sistemul dezactivează butonul pentru a preveni dubla trimitere și afișează un indicator de încărcare (spinner sau text "Se calculează...").

Pasul 9: Frontend-ul trimite o cerere HTTP POST la backend, la endpoint-ul /api/trips/optimize, cu corpul cererii în format JSON: { origin: "București", destination: "Brașov", date: "2026-05-25", time: "10:00", criteria: "time" }.

Pasul 10: Backend-ul primește cererea și validează datele. Validează că toate câmpurile obligatorii sunt prezente și că formatul este corect. Dacă validarea eșuează, returnează un răspuns cu codul HTTP 400 și un mesaj de eroare.

Pasul 11: Backend-ul interoghează baza de date pentru datele de trafic relevante. Execută interogarea SQL: SELECT zone, congestion_level, average_speed FROM traffic_data WHERE zone IN ('București', 'Brașov', 'Ploiești') AND timestamp BETWEEN '2026-05-25 09:00:00' AND '2026-05-25 11:00:00'.

Pasul 12: Backend-ul aplică algoritmul de optimizare multi-criterială. Pentru criteriul "Timp minim", ponderile sunt: timp = 0.5, cost = 0.2, trafic = 0.2, distanță = 0.1. Algoritmul calculează scorul pentru fiecare rută posibilă și selectează ruta cu scorul cel mai mic.

Pasul 13: Backend-ul construiește răspunsul JSON care conține ruta (listă de coordonate geografice pentru a fi trasate pe hartă), timpul estimat (150 minute), costul estimat (45 lei), distanța (180 km) și nivelul de trafic (mediu). Trimite răspunsul cu codul HTTP 200.

Pasul 14: Frontend-ul primește răspunsul și ascunde indicatorul de încărcare. Procesează lista de coordonate și trasează ruta pe hartă folosind Leaflet.js. Adaugă un marker verde la origine, un marker roșu la destinație și o linie albastră care leagă punctele rutei.

Pasul 15: Frontend-ul afișează rezultatele în panoul dedicat: "Timp estimat: 2 ore 30 minute", "Cost estimat: 45 lei", "Distanță: 180 km", "Nivel trafic: Mediu".

Pasul 16: Sistemul afișează butonul "Salvează în istoric" dar îl menține dezactivat sau afișează un tooltip care spune "Trebuie să fii autentificat pentru a salva rute". Sistemul afișează și un link "Autentifică-te sau înregistrează-te pentru a salva această rută".

Cazuri de eroare posibile:

Dacă utilizatorul lasă câmpurile obligatorii necompletate și apasă butonul, sistemul nu trimite cererea și afișează mesaje de eroare lângă fiecare câmp: "Câmp obligatoriu".

Dacă backend-ul nu răspunde în 10 secunde, frontend-ul afișează mesajul "Serverul nu răspunde. Vă rugăm să încercați mai târziu".

Dacă API-ul OpenStreetMap este indisponibil, harta poate să nu se încarce corect. Sistemul afișează un mesaj de avertizare în consola dezvoltatorului, dar ruta poate fi afișată în continuare într-un format text simplu.

Scenariul 2: Utilizator autentificat își salvează o rută în istoric

Clasă utilizator: Utilizator autentificat
Scop: Salvarea unei rute calculate pentru a o putea accesa ulterior
Mod operațional: Normal

Descrierea scenariului: Un utilizator care are cont pe platformă se autentifică, calculează o rută și o salvează în istoricul personal.

Fluxul pas cu pas:

Pasul 1: Utilizatorul accesează pagina Cont și introduce emailul "user@example.com" și parola "parola123". Apasă butonul "Autentificare".

Pasul 2: Frontend-ul trimite o cerere POST la /api/auth/login cu credentialele. Backend-ul verifică dacă emailul există în tabela users și compară parola introdusă cu hash-ul stocat folosind bcrypt.compare.

Pasul 3: Dacă credentialele sunt corecte, backend-ul creează o sesiune pentru utilizator și trimite un răspuns cu succes. Frontend-ul actualizează interfața pentru a reflecta starea autentificată (afișează numele utilizatorului în antet, schimbă link-ul "Cont" în "Profilul meu").

Pasul 4: Utilizatorul navighează la pagina Planificare, completează formularul (București -> Constanța, data 26-05-2026, ora 08:00, criteriu "Cost minim") și apasă "Calculează ruta". Sistemul calculează ruta similar cu Scenariul 1.

Pasul 5: După ce ruta este afișată, sistemul afișează butonul "Salvează în istoric" activat (spre deosebire de cazul vizitatorului). Utilizatorul apasă acest buton.

Pasul 6: Frontend-ul trimite o cerere POST la /api/trips/save cu datele rutei: { origin: "București", destination: "Constanța", date: "2026-05-26", cost: 65, duration: 180 }. Cererea include cookie-ul de sesiune pentru autentificare.

Pasul 7: Backend-ul verifică sesiunea și extrage user_id-ul utilizatorului autentificat. Apoi execută interogarea SQL: INSERT INTO trips (user_id, origin, destination, date, cost, duration) VALUES ($1, $2, $3, $4, $5, $6) RETURNING id.

Pasul 8: Backend-ul returnează un răspuns de succes cu id-ul înregistrării nou create. Frontend-ul afișează un mesaj de confirmare: "Rută salvată cu succes în istoric".

Pasul 9: Utilizatorul navighează la pagina Istoric. Frontend-ul trimite o cerere GET la /api/trips/history. Backend-ul execută SELECT * FROM trips WHERE user_id = $1 ORDER BY created_at DESC. Returnează lista rutelor salvate.

Pasul 10: Frontend-ul afișează tabelul cu rutele salvate, inclusiv ruta tocmai adăugată (București - Constanța). Utilizatorul poate vedea toate rutele anterioare.

Caz de eroare - salvare duplicat: Dacă utilizatorul încearcă să salveze aceeași rută de mai multe ori, sistemul nu împiedică acest lucru (permite duplicate în istoric). Dacă utilizatorul dorește să șteargă o rută, apasă butonul "Șterge" din tabel, sistemul confirmă acțiunea și trimite o cerere DELETE la /api/trips/:id.

Scenariul 3: Administratorul vizualizează statisticile platformei

Clasă utilizator: Administrator
Scop: Vizualizarea statisticilor globale și gestionarea utilizatorilor
Mod operațional: Normal

Descrierea scenariului: Un administrator se autentifică și accesează panoul de administrare pentru a vedea numărul de utilizatori și călătoriile pe lună.

Fluxul pas cu pas:

Pasul 1: Administratorul se autentifică folosind un cont care are câmpul is_admin setat la TRUE în baza de date. Procesul de autentificare este identic cu scenariul 2.

Pasul 2: După autentificare, sistemul verifică valoarea is_admin pentru utilizatorul curent. Deoarece valoarea este TRUE, sistemul afișează un link suplimentar în meniul de navigare: "Panou Admin".

Pasul 3: Administratorul dă click pe link-ul "Panou Admin". Frontend-ul trimite o cerere GET la /api/admin/stats.

Pasul 4: Backend-ul verifică din nou că utilizatorul are is_admin = TRUE (verificare de securitate, nu doar UI). Dacă nu, returnează codul HTTP 403 Forbidden.

Pasul 5: Backend-ul execută mai multe interogări SQL pentru a colecta statisticile. Interogarea pentru numărul total de utilizatori: SELECT COUNT*() FROM users. Interogarea pentru numărul total de călătorii: SELECT COUNT()* FROM trips. Interogarea pentru călătorii pe lună: SELECT DATE_TRUNC('month', created_at) as month, COUNT*() FROM trips GROUP BY month ORDER BY month. Interogarea pentru top destinații: SELECT destination, COUNT()* as count FROM trips GROUP BY destination ORDER BY count DESC LIMIT 3.

Pasul 6: Backend-ul agregează toate aceste date într-un obiect JSON și le trimite frontend-ului.

Pasul 7: Frontend-ul afișează în partea de sus a paginii Admin carduri cu numerele: "Total utilizatori: 47", "Total călătorii: 156".

Pasul 8: Frontend-ul afișează un grafic Chart.js de tip bară pentru călătoriile pe lună, cu lunile pe axa X și numărul de călătorii pe axa Y.

Pasul 9: Frontend-ul afișează un tabel cu top 3 destinații: locul 1 - Brașov (42 călătorii), locul 2 - Constanța (38 călătorii), locul 3 - Cluj-Napoca (25 călătorii).

Caz de eroare - acces neautorizat: Dacă un utilizator autentificat normal (is_admin = FALSE) încearcă să acceseze direct endpoint-ul /api/admin/stats (de exemplu, folosind un instrument precum Postman), backend-ul returnează codul HTTP 403 Forbidden, iar frontend-ul afișează un mesaj "Acces interzis".

Scenariul 4: Algoritmul de optimizare multi-criterial (detaliat tehnic)

Clasă utilizator: Sistem (backend)
Scop: Calcularea rutei optime pe baza criteriilor personalizate ale utilizatorului
Mod operațional: Procesare internă

Descrierea scenariului: Acest scenariu descrie în detaliu pașii algoritmului de optimizare multi-criterială care reprezintă inovația proiectului. Algoritmul primește ca intrare rutele posibile și returnează ruta cu scorul minim în funcție de ponderile specifice criteriului ales.

Fluxul pas cu pas:

Pasul 1: Backend-ul primește de la frontend un obiect JSON cu parametrii: origin (șir de caractere), destination (șir de caractere), date (dată), time (oră), criteria (unul dintre valorile: "time", "cost", "traffic", "distance", "eco").

Pasul 2: Backend-ul construiește graful rutelor posibile între origine și destinație. Graful este reprezentat ca o listă de noduri (orașe) și muchii (drumuri între orașe). Pentru proiectul demonstrativ, graful este predefinit pentru principalele orașe din România (aproximativ 30-40 de noduri).

Pasul 3: Pentru fiecare rută posibilă identificată (prin algoritmul Dijkstra sau A* adaptat), sistemul calculează patru valori brute: timp_total (în minute), cost_total (în lei), nivel_trafic (procent între 0 și 100, extras din baza de date), distanta_totală (în kilometri).

Pasul 4: Sistemul calculează valorile minime și maxime pentru fiecare criteriu pe întregul set de rute posibile. De exemplu, timp_min = 120 minute, timp_max = 300 minute; cost_min = 40 lei, cost_max = 120 lei; trafic_min = 10%, trafic_max = 80%; distanta_min = 150 km, distanta_max = 250 km.

Pasul 5: Sistemul normalizează fiecare valoare brută la intervalul [0, 1] folosind formula de normalizare Min-Max: valoare_normalizata = (valoare_brută - valoare_min) / (valoare_max -

valoare_min). Valorile mai mici înseamnă rute mai bune (timp mai scurt, cost mai mic, trafic mai redus, distanță mai scurtă).

Pasul 6: Sistemul determină ponderile în funcție de criteriul selectat de utilizator. Pentru criteriul "Timp minim", ponderile sunt: w_time = 0.5, w_cost = 0.2, w_traffic = 0.2, w_distance = 0.1. Pentru criteriul "Cost minim", ponderile sunt: w_time = 0.2, w_cost = 0.5, w_traffic = 0.2, w_distance = 0.1. Pentru criteriul "Trafic redus", ponderile sunt: w_time = 0.2, w_cost = 0.2, w_traffic = 0.5, w_distance = 0.1. Pentru criteriul "Distanță minimă", ponderile sunt: w_time = 0.2, w_cost = 0.1, w_traffic = 0.1, w_distance = 0.6. Pentru criteriul "Eco-friendly", ponderile pot fi w_time = 0.2, w_cost = 0.3, w_traffic = 0.3, w_distance = 0.2 (promovează rutele cu trafic redus și distanță moderată).

Pasul 7: Pentru fiecare rută posibilă, sistemul calculează scorul final ca suma ponderată: scor = (w_time * timp_normalizat) + (w_cost * cost_normalizat) + (w_traffic * trafic_normalizat) + (w_distance * distanta_normalizat).

Pasul 8: Sistemul selectează ruta cu scorul cel mai mic (minimizarea scorului). Exemplu numeric pentru două rute între București și Brașov:

Ruta A: timp=150 min, cost=45 lei, trafic=30%, distanta=170 km. După normalizare: timp_n=0.2, cost_n=0.1, trafic_n=0.3, dist_n=0.2. Pentru criteriul "Timp minim" (ponderile 0.5,0.2,0.2,0.1), scor = 0.5*0.2 + 0.2*0.1 + 0.2*0.3 + 0.1*0.2 = 0.1 + 0.02 + 0.06 + 0.02 = 0.20.

Ruta B: timp=180 min, cost=35 lei, trafic=50%, distanta=200 km. Normalizare: timp_n=0.4, cost_n=0, trafic_n=0.6, dist_n=0.4. Scor = 0.5*0.4 + 0.2*0 + 0.2*0.6 + 0.1*0.4 = 0.2 + 0 + 0.12 + 0.04 = 0.36.

Ruta A are scor mai mic (0.20 < 0.36), deci este selectată.

Pasul 9: Sistemul construiește răspunsul JSON care conține informațiile rutei selectate: lista de puncte geografice (pentru trasarea pe hartă), timpul brut (150 minute), costul brut (45 lei), distanța brută (170 km), nivelul de trafic (30%). Trimite răspunsul la frontend.

Scenariul 5: Utilizatorul încearcă să acceseze o resursă protejată fără autentificare

Clasă utilizator: Vizitator (neautentificat)
Scop: Testarea mecanismului de securitate
Mod operațional: Eroare de autorizare

Descrierea scenariului: Un vizitator încearcă să acceseze direct pagina Istoric sau un endpoint API care necesită autentificare.

Fluxul pas cu pas:

Pasul 1: Vizitatorul este pe platformă, dar nu s-a autentificat. Nu vede link-ul "Istoric" în meniul de navigare (deoarece frontend-ul ascunde acest link pentru utilizatorii neautentificați).

Pasul 2: Vizitatorul încearcă să acceseze manual adresa /istoric în browser (https://localhost:3000/istoric). Frontend-ul (routing-ul pe client) detectează că utilizatorul nu este autentificat și redirecționează automat către pagina Cont, afișând un mesaj: "Vă rugăm să vă autentificați pentru a accesa această pagină".

Pasul 3: Vizitatorul încearcă să trimită direct o cerere API către /api/trips/history folosind un instrument precum Developer Tools din browser. Backend-ul primește cererea, dar nu găsește un cookie de sesiune valid sau sesiunea nu conține user_id.

Pasul 4: Backend-ul returnează codul HTTP 401 Unauthorized cu corpul JSON: { success: false, error: { code: "UNAUTHORIZED", message: "Trebuie să fii autentificat pentru a accesa această resursă" } }.

Pasul 5: Frontend-ul primește acest răspuns și afișează un mesaj de eroare în consolă, dar nu arată nimic utilizatorului pentru a nu-i strica experiența (în mod normal, utilizatorul nu ar ajunge în această situație deoarece UI-ul nu îi permite).

Caz similar pentru administrator: Dacă un utilizator autentificat normal (is_admin = FALSE) încearcă să acceseze endpoint-ul /api/admin/users, backend-ul returnează codul HTTP 403 Forbidden. Frontend-ul afișează un mesaj "Acces interzis. Nu aveți permisiunea de a accesa această pagină."

7. Proiectare de detaliu

Această secțiune furnizează informațiile necesare echipei de dezvoltare pentru a construi și integra efectiv componentele hardware și software ale sistemului TripSmart Agency.

7.1 Proiectare hardware de detaliu

Componente hardware utilizate

Sistemul TripSmart Agency utilizează doar componente hardware standard, disponibile pe orice calculator personal destinat dezvoltării software. Nu sunt necesare componente hardware specializate.

Stația de dezvoltare (calculator personal)

Aceasta este singura componentă hardware necesară. Furnizorul este la alegerea utilizatorului (Dell, HP, Lenovo, Apple, etc.). Specificațiile minime recomandate sunt: procesor Intel Core i5 (sau AMD Ryzen 5) cu frecvență minimă 2.5 GHz, memorie RAM 8 GB DDR4, stocare SSD 256 GB, placă de rețea Ethernet sau Wi-Fi pentru acces la internet.

Cerințele de intrare de alimentare sunt standard: 220V AC, 50-60 Hz, consum maxim aproximativ 100-150 W pentru un laptop sau 300-400 W pentru un desktop. Nu există cerințe speciale privind impedanțe sau stări logice ale semnalului, deoarece toate componentele sunt digitale și utilizează interfețe standard.

Conectorii utilizați sunt cei standard: USB pentru tastatură și mouse, HDMI sau DisplayPort pentru monitor, Ethernet RJ-45 pentru rețea (sau Wi-Fi intern). Spațiul de stocare necesar este de aproximativ 1 GB pentru codul sursă, baza de date și dependențe. Procesorul trebuie să suporte virtualizarea (nu este obligatoriu dar util) și setul de instrucțiuni x86-64.

Dispozitive periferice

Monitorul este necesar pentru afișarea interfeței utilizatorului și a codului. Rezoluția recomandată este de minimum 1366x768 pixeli, dar 1920x1080 este preferabilă. Tastatura și mouse-ul sunt standard, pot fi orice model compatibil PC (USB sau wireless). Conexiunea la internet este necesară doar pentru accesul la API-ul OpenStreetMap și pentru descărcarea pachetelor npm. O conexiune de minimum 10 Mbps este suficientă.

Reprezentarea grafică a componentelor hardware

Toate componentele hardware sunt integrate într-o singură unitate fizică (calculatorul personal). Nu există componente distribuite sau separate geografic. Poziționarea relativă este irelevantă deoarece toate sunt în aceeași carcasă sau conectate prin cabluri scurte.

7.2 Proiectare software de deatliu

Această secțiune descrie în detaliu principalele servicii software ale sistemului TripSmart Agency.

Serviciul 1: AuthenticationService

Identificator serviciu: SVC-AUTH-01
Clasificare: Serviciu de securitate / autentificare
Limbaj: JavaScript (Node.js)
Estimare SLOC: 150-200 linii

Definiție: Serviciul de autentificare gestionează înregistrarea de conturi noi, autentificarea utilizatorilor existenți și gestionarea sesiunilor. Acesta asigură că doar utilizatorii care își dovedesc identitatea pot accesa resursele protejate.

Cerințe satisfăcute: REQ-3.1 (înregistrare), REQ-3.2 (autentificare), REQ-3.3 (gestionare profil)

Structuri de date interne:

```
const userSchema = {
id: number,
name: string,
email: string,
passwordHash: string,
createdAt: Date,
isAdmin: boolean
};
```

Sesiunea utilizatorului este stocată în memoria serverului (express-session) și conține user_id și isAdmin.

Constrângeri:

Parolele trebuie să aibă minimum 6 caractere (validare pe frontend și backend). Emailurile trebuie să fie unice în baza de date. Hash-ul parolei este generat cu bcrypt cu factor de cost 10. Timpul maxim de viață al sesiunii este de 24 de ore. Precondiții: pentru înregistrare, toate câmpurile trebuie să fie prezente și valide. Pentru autentificare, emailul trebuie să existe și parola

să corespundă hash-ului. Postcondiții: după autentificare reușită, sesiunea conține user_id și is_admin.

Compoziție: Serviciul utilizează subserviciul bcrypt pentru criptarea și verificarea parolelor și subserviciul express-session pentru gestionarea sesiunilor.

Utilizatori/Interacțiuni:

Serviciul este utilizat de AuthController (pentru procesarea cererilor HTTP) și de AuthMiddleware (pentru verificarea autentificării). AuthController apelează AuthenticationService pentru a înregistra sau autentifica utilizatori. AuthMiddleware apelează serviciul pentru a verifica dacă o sesiune este validă. Efecte secundare: la înregistrare, se creează o înregistrare în tabela users. La autentificare, se actualizează sesiunea.

Procesare:

Pentru înregistrare, serviciul primește name, email, password. Validează că emailul nu există deja în baza de date. Generează hash-ul parolei folosind bcrypt.hash(password, 10). Inserează înregistrarea în tabela users. Returnează obiectul utilizator (fără parolă).

Pentru autentificare, serviciul primește email, password. Caută utilizatorul în baza de date după email. Dacă nu există, returnează eroare. Compară parola primită cu hash-ul stocat folosind bcrypt.compare(password, hash). Dacă nu corespund, returnează eroare. Inițializează sesiunea cu user_id și isAdmin. Returnează obiectul utilizator (fără parolă).

Complexitatea temporală este O(1) pentru operațiile de bază de date (cu indici) și O(k) pentru bcrypt (unde k este factorul de cost). Concurența este gestionată de pool-ul de conexiuni PostgreSQL.

Interfețe/Exporturi:

register(name, email, password) -> userObject sau error
login(email, password, session) -> userObject sau error
logout(session) -> void
getUserById(userId) -> userObject sau null
changePassword(userId, oldPassword, newPassword) -> boolean

Serviciul 2: OptimizationService

Identificator serviciu: SVC-OPT-01
Clasificare: Serviciu de calcul / algoritmic
Limbaj: JavaScript (Node.js)
Estimare SLOC: 200-250 linii

Definiție: Serviciul de optimizare implementează algoritmul multi-criterial de scoring ponderat pentru determinarea rutei optime între două locații. Acesta reprezintă inovația principală a proiectului de licență.

Cerințe satisfăcute: REQ-1.3 (algoritm de optimizare), REQ-1.1 (formular rută), REQ-1.2 (selectare criterii)

Structuri de date interne:

```
const routeGraph = {
nodes: ["București", "Brașov", "Ploiești", ...],
edges: [
{ from: "București", to: "Ploiești", time: 60, cost: 25, distance: 60 },
{ from: "Ploiești", to: "Brașov", time: 90, cost: 30, distance: 90 },
...
]
};

const criteriaWeights = {
time: { time: 0.5, cost: 0.2, traffic: 0.2, distance: 0.1 },
cost: { time: 0.2, cost: 0.5, traffic: 0.2, distance: 0.1 },
traffic: { time: 0.2, cost: 0.2, traffic: 0.5, distance: 0.1 },
distance: { time: 0.2, cost: 0.1, traffic: 0.1, distance: 0.6 },
eco: { time: 0.2, cost: 0.3, traffic: 0.3, distance: 0.2 }
};
```

Constrângeri:

Graful rutelor este predefinit pentru principalele 30-50 de orașe din România. Algoritmul nu suportă rute care trec prin mai mult de 10 noduri intermediare din cauza complexității. Timpul de execuție trebuie să fie sub 3 secunde pentru orice pereche origine-destinație din graful predefinit. Precondiții: origin și destination trebuie să existe în graful de noduri. criteria trebuie să fie una dintre valorile valide ("time", "cost", "traffic", "distance", "eco"). Postcondiții: rezultatul conține ruta optimă (lista de noduri) și valorile agregate (timp, cost, trafic, distanță).

Compoziție: Serviciul utilizează un algoritm de căutare a drumurilor (Dijkstra sau BFS) pentru a genera rutele posibile între origine și destinație. Apoi aplică funcția de scoring ponderat pe fiecare rută generată.

Utilizatori/Interacțiuni:

Serviciul este utilizat exclusiv de TripController la endpoint-ul POST /api/trips/optimize. Nu are efecte secundare asupra altor părți ale sistemului (nu modifică baza de date). Interacționează indirect cu TrafficService pentru a obține datele de trafic pentru zonele și intervalele orare relevante.

Procesare:

Algoritmul parcurge următorii pași:

Generarea rutelor posibile folosind un algoritm BFS modificat care explorează graful de la origine la destinație. Se limitează adâncimea de căutare la maximum 10 noduri pentru a menține performanța.

Pentru fiecare rută generată, se calculează valorile brute: timp_total = suma(timp_muchii), cost_total = suma(cost_muchii), distanta_totala = suma(distanta_muchii). Traficul este determinat prin interogarea bazei de date pentru fiecare segment și calcularea mediei ponderate.Se calculează valorile minime și maxime pentru fiecare criteriu pe întregul set de rute.

Se normalizează fiecare valoare brută la intervalul [0,1] folosind formula: (valoare - min) / (max - min).

Se aplică ponderile corespunzătoare criteriului selectat și se calculează scorul final.

Se selectează ruta cu scorul minim.Se construiește obiectul rezultat cu ruta (lista de noduri) și valorile agregate.

Complexitatea temporală este O(n * m) unde n este numărul de rute generate și m este numărul de criterii (4). Pentru un graf cu 50 de noduri, timpul de execuție este sub 100 ms. Complexitatea spațială este O(n) pentru stocarea rutelor intermediare.

Interfețe/Exporturi:

calculateOptimalRoute(origin, destination, criteria, date, time) -> routeResult
getAvailableDestinations(origin) -> arrayOfDestinations
getRouteDetails(routeId) -> routeDetails

Serviciul 3: TrafficService

Identificator serviciu: SVC-TRF-01
Clasificare: Serviciu de date / analiză
Limbaj: JavaScript (Node.js)
Estimare SLOC: 100-150 linii

Definiție: Serviciul de trafic gestionează colectarea, stocarea și agregarea datelor de trafic. Acesta furnizează datele necesare algoritmului de optimizare și pentru generarea graficelor statistice.

Cerințe satisfăcute: REQ-2.1 (colectare date trafic), REQ-2.2 (generare grafice), REQ-2.3 (predicție trafic)

Structuri de date interne:

```
const trafficDataSchema = {
id: number,
zone: string,
timestamp: Date,
congestionLevel: number, // 0-100
averageSpeed: number // km/h
};
```

Constrângeri:

Datele de trafic sunt generate sintetic pentru demonstrație. Interogările trebuie să fie parametrizate pentru a preveni SQL injection. Timpul de răspuns pentru interogările agregate trebuie să fie sub 500 ms. Precondiții: zone trebuie să fie un șir nevid, congestionLevel între 0 și 100, averageSpeed >= 0. Postcondiții: datele sunt stocate persistent în baza de date.

Compoziție: Serviciul utilizează modelul TrafficModel pentru operațiile CRUD pe baza de date și ChartService (frontend) pentru generarea graficelor.

Utilizatori/Interacțiuni:

Serviciul este utilizat de OptimizationService (pentru a obține date de trafic pentru rute) și de TrafficController (pentru endpoint-urile API). Efecte secundare: scrierea în tabela traffic_data la inițializarea sistemului.

Procesare:

Pentru obținerea datelor de trafic pentru o rută, serviciul primește o listă de zone și un interval orar. Execută interogarea SQL SELECT zone, AVG(congestion_level) as avg_congestion, AVG(average_speed) as avg_speed FROM traffic_data WHERE zone = ANY($1) AND timestamp BETWEEN $2 AND $3 GROUP BY zone. Returnează un obiect mapat pe zone cu valorile agregate.

Pentru generarea statisticilor pentru grafice, serviciul execută interogări similare grupate pe interval orar sau pe zonă.

Interfețe/Exporturi:

```
getTrafficForZones(zones, startTime, endTime) -> trafficDataMap
getTrafficStatsByHour(zone, date) -> hourlyStats
getTrafficStatsByZone(startDate, endDate) -> zoneStats
seedTrafficData() -> void (pentru inițializare)
```

7.3 Proiectare detaliată de securitate

Autentificare

Sistemul utilizează autentificare bazată pe sesiuni HTTP. Mecanismul este implementat astfel:

La înregistrare, parola utilizatorului este criptată folosind bcrypt cu un factor de cost de 10. Hash-ul rezultat este stocat în coloana password_hash a tabelei users. Parola originală nu este păstrată nicăieri.

La autentificare, utilizatorul trimite emailul și parola. Backend-ul caută utilizatorul după email. Dacă există, folosește bcrypt.compare pentru a verifica dacă parola trimisă corespunde hash-ului stocat. Dacă da, creează o sesiune folosind express-session. Sesiunea stochează user_id și is_admin.

Cookie-ul de sesiune are flag-ul httpOnly setat la true (nu poate fi accesat de JavaScript), secure în producție (doar HTTPS), sameSite='lax'.

Autorizare

Autorizarea se face pe bază de roluri determinate de câmpul is_admin. Există două niveluri de autorizare.

Nivelul utilizator autentificat: poate accesa endpoint-urile /api/trips/history, /api/trips/save, /api/trips/:id (DELETE), /api/user/profile. Verificarea se face prin middleware-ul isAuthenticated care verifică existența user_id în sesiune.

Nivelul administrator: poate accesa toate endpoint-urile utilizatorului autentificat plus /api/admin/*. Verificarea se face prin middleware-ul isAdmin care, pe lângă verificarea autentificării, verifică și că is_admin din sesiune este TRUE.

Jurnalizare și auditare

Sistemul implementează un middleware de logging care înregistrează pentru fiecare cerere HTTP: timestamp-ul, metoda HTTP, URL-ul, codul de stare HTTP, user_id (dacă utilizatorul este autentificat), adresa IP a clientului.

Logurile sunt scrise în consola serverului în dezvoltare. În producție, pot fi scrise într-un fișier folosind pachete precum winston. Evenimentele de securitate înregistrate includ: încercări de autentificare eșuate, accesări de resurse fără autorizație, încercări de acces la endpoint-uri administrative de către utilizatori neautorizați.

Criptare

Singurele date criptate stocate sunt parolele utilizatorilor, folosind bcrypt. bcrypt este un algoritm de hash care include automat un salt pentru a preveni atacurile cu tabele rainbow. Nu se utilizează criptare pentru alte date deoarece nu sunt considerate sensibile.

Pentru transmiterea datelor, în mediul de dezvoltare se folosește HTTP simplu (fără criptare) deoarece totul rulează pe localhost. În cazul unui deployment în producție, se va folosi HTTPS cu un certificat SSL/TLS valid.

Utilizarea porturilor de rețea

Sistemul utilizează următoarele porturi de rețea în configurația implicită. Portul 3000 pentru serverul Node.js (Express) - HTTP. Portul 5432 pentru serverul PostgreSQL (TCP). Portul 443 pentru conexiunile la API-ul OpenStreetMap (HTTPS, inițiate de browser). Nu sunt expuse alte porturi către rețea.

Detectarea și prevenirea intruziunilor

În contextul unui proiect academic care rulează pe localhost, nu sunt implementate măsuri speciale de detectare a intruziunilor. Singurele măsuri de precauție sunt utilizarea interogărilor parametrizate pentru prevenirea SQL injection și validarea inputurilor utilizatorului pe backend.

7.4 Proiectare de detaliu pentru performanța sistemului

Cerințe și estimări de capacitate și volum

Capacitatea estimată a sistemului în scenariul de demonstrație este de maximum 30 de utilizatori simultani și maximum 10 cereri pe secundă. Volumul de date estimat este de maximum 100 de utilizatori în baza de date, maximum 500 de călătorii salvate, maximum 10.000 de înregistrări de trafic.

Așteptări de performanță

Timpul de încărcare a unei pagini (primul octet) trebuie să fie sub 500 ms. Timpul de răspuns pentru calculul rutei (endpoint-ul /api/trips/optimize) trebuie să fie sub 3 secunde. Timpul de răspuns pentru interogările simple (autentificare, istoric) trebuie să fie sub 200 ms.

Cerințe de disponibilitate

În mediul de dezvoltare, disponibilitatea necesară este de 95% (orele de lucru). Nu există cerințe de funcționare 24/7. Punctele unice de eșec sunt: serverul Node.js (dacă se oprește, întreaga aplicație devine indisponibilă), baza de date PostgreSQL (dacă se oprește, operațiile de citire/scriere eșuează).

Proiectare de performanță

Pentru a îndeplini cerințele de capacitate și performanță, s-au luat următoarele măsuri. Indexarea coloanelor folosite frecvent în interogări WHERE (email pe users, user_id pe trips, zone și timestamp pe traffic_data). Limitarea grafului de rute la maximum 50 de noduri pentru a menține algoritmul de optimizare rapid. Implementarea unui cache simplu în memorie pentru rezultatele frecvente ale calculului rutelor (opțional). Utilizarea unui pool de conexiuni la baza de date pentru a evita suprasolicitarea.

Proiectare de fiabilitate

Pentru a crește fiabilitatea, s-au implementat următoarele mecanisme. Mecanism de retry pentru cererile către API-ul extern OpenStreetMap (maxim 3 încercări cu backoff exponențial). Gestionarea centralizată a erorilor cu răspunsuri JSON consistente. În cazul în care baza de date nu răspunde, serverul returnează un mesaj de eroare prietenos.

Proiectare de backup și recuperare

Backup-ul bazei de date se face folosind pg_dump. Comanda recomandată este pg_dump -U postgres -d tripsmart_db > backup.sql. Backup-ul se face săptămânal. Pentru recuperare, se folosește comanda psql -U postgres -d tripsmart_db < backup.sql. Codul sursă este backup-at prin Git (repository pe GitHub).

Puncte unice de eșec

Singurul punct unic de eșec în arhitectura actuală este serverul Node.js. Dacă acesta se oprește, întreaga aplicație devine indisponibilă. Deoarece proiectul este demonstrativ și rulează pe localhost, nu se implementează soluții de clustering sau load balancing. În caz de oprire, simpla repornire a serverului restabilește funcționalitatea.

7.5 Proiectare detaliată a comunicațiilor interne (între componente)

Numărul de servere și clienți

În configurația standard de dezvoltare și demonstrație, există un singur server care găzduiește atât aplicația Node.js cât și baza de date PostgreSQL. Numărul de clienți (browsere) este variabil, estimat la maximum 30 simultan. Toate componentele sunt conectate printr-o singură rețea de zonă locală virtuală (localhost).

Specificații pentru cerințele de sincronizare

Comunicațiile sunt asincrone și bazate pe evenimente (Node.js event loop). Nu există cerințe stricte de sincronizare în timp real. Cererile HTTP sunt procesate în ordinea

sosirii, dar fără garanții stricte de ordine. Baza de date PostgreSQL utilizează mecanisme standard de concurență (MVCC) și nu necesită sincronizare suplimentară.

Formatul datelor schimbate între componente

Între browser și backend, formatul este JSON (JavaScript Object Notation). Exemplu de cerere pentru calcul rută:

{
"origin": "București",
"destination": "Brașov",
"date": "2026-05-25",
"time": "10:00",
"criteria": "time"
}

Exemplu de răspuns pentru calcul rută:{

"success": true,
"data": {
"route": [
{ "lat": 44.4268, "lng": 26.1025 },
{ "lat": 44.9425, "lng": 25.9824 },
{ "lat": 45.6579, "lng": 25.6012 }
],
"totalTime": 150,
"totalCost": 45,
"totalDistance": 180,
"trafficLevel": 30
}
}

Între backend și baza de date, formatul este SQL (Structurat Query Language). Exemplu: SELECT * FROM users WHERE email = 'user@example.com'.

Reprezentarea grafică a conectivității

Browser (client) -(HTTP, port 3000)-Server Node.js (backend)Server Node.js -(TCP, port 5432)-> PostgreSQL (baza de date)

Browser -(HTTPS, port 443)-> OpenStreetMap API (extern)

Toate componentele sunt co-locate pe aceeași mașină fizică. Distanțele sunt neglijabile (câțiva centimetri în interiorul aceleiași carcase). Nu sunt necesare cabluri de rețea externe în afara conexiunii la internet pentru accesul la OpenStreetMap.

Topologia LAN

În configurația de dezvoltare, toate componentele rulează pe aceeași mașină, deci nu există o rețea LAN fizică în sens tradițional. Adresele IP utilizate sunt 127.0.0.1 (localhost) pentru toate comunicațiile interne. Nu sunt necesare switch-uri, routere sau firewall-uri hardware

8. Controale pentru verificarea integrității sistemului

- Această secțiune furnizează specificațiile de proiectare pentru controalele minime necesare asigurării integrității și securității sistemului TripSmart Agency.

8.1 Securitate internă pentru restricționarea accesului la datele critice

- **Clasificarea datelor critice**

- În cadrul sistemului TripSmart Agency, următoarele date sunt considerate critice și beneficiază de restricții de acces:

- Datele de autentificare: hash-urile parolelor utilizatorilor (coloana password_hash din tabela users). Aceste date nu trebuie să fie accesibile nici măcar administratorilor în formă decriptată. Accesul este permis doar algoritmului bcrypt în timpul autentificării, care compară parola introdusă cu hash-ul fără a-l expune.

- Datele personale: numele și emailurile utilizatorilor. Aceste date sunt accesibile utilizatorului propriu (pentru vizualizarea și editarea propriului profil) și administratorilor (pentru gestionarea conturilor). Alți utilizatori nu au acces la aceste date.

- Datele operaționale: călătoriile salvate (tabela trips). Aceste date sunt accesibile doar utilizatorului care le-a creat și administratorilor. Accesul se face pe baza user_id din sesiune.

- Datele de trafic (tabela traffic_data) nu sunt considerate critice și sunt accesibile public (utilizatori autentificați și neautentificați).

- **Mecanisme de restricționare a accesului**

- La nivelul bazei de date, se utilizează un singur utilizator PostgreSQL (tripsmart_user) care are privilegii de SELECT, INSERT, UPDATE, DELETE pe toate tabelele. Restricționarea fină se face la nivelul aplicației, nu la nivelul bazei de date, deoarece aplicația este singurul client al bazei de date.

- La nivelul aplicației backend, se implementează două middleware-uri de control al accesului.

- Middleware-ul isAuthenticated verifică dacă sesiunea curentă conține un user_id valid. Dacă nu, returnează codul HTTP 401 Unauthorized. Acest middleware este aplicat tuturor endpoint-urilor care necesită autentificare: POST /api/trips/save, GET /api/trips/history, DELETE /api/trips/:id, PUT /api/user/profile.

- Middleware-ul isAdmin verifică dacă utilizatorul autentificat are flag-ul is_admin setat la TRUE. Dacă nu, returnează codul HTTP 403 Forbidden. Acest middleware este aplicat endpoint-urilor de administrare: GET /api/admin/users, GET /api/admin/stats, DELETE /api/admin/users/:id.

- În plus, în controlerele care accesează date specifice unui utilizator (de exemplu, ștergerea unei călătorii), se verifică suplimentar că resursa accesată aparține utilizatorului autentificat. Interogarea SQL include condiția AND user_id = $currentUserId, prevenind accesul la datele altor utilizatori chiar dacă middleware-ul de autentificare a fost ocolit.

- La nivelul frontend-ului, link-urile către pagini protejate (Istoric, Admin) sunt afișate condiționat pe baza stării de autentificare și a privilegiilor. Aceasta

este doar o măsură de confort, nu una de securitate - verificările reale se fac pe backend.

- ## 8.2 Proceduri de audit

- **Obiectivele auditării**

- Sistemul TripSmart Agency implementează proceduri de audit pentru a înregistra evenimentele semnificative din punct de vedere al securității și operațiunilor. Aceste înregistrări sunt utilizate pentru depanare, pentru identificarea încercărilor de acces neautorizat și pentru conformitatea cu cerințele academice.

- **Evenimente auditate**

- Următoarele categorii de evenimente sunt înregistrate în jurnalele de audit:

- Evenimente de autentificare: încercări de autentificare reușite și eșuate, inclusiv emailul utilizat și adresa IP sursă. Evenimentele eșuate sunt înregistrate cu nivel WARN, cele reușite cu nivel INFO.

- Evenimente de autorizare: încercări de acces la resurse protejate fără permisiuni corespunzătoare (cod HTTP 403). Se înregistrează user_id (dacă există), URL-ul accesat, metoda HTTP și timestamp-ul.

- Evenimente de manipulare a datelor critice: crearea, modificarea sau ștergerea unui cont de utilizator (inclusiv schimbarea rolului is_admin). Se înregistrează administratorul care a efectuat acțiunea, utilizatorul vizat, tipul

acțiunii și timestamp-ul. De asemenea, modificarea parolei unui utilizator (inițiată de utilizatorul însuși sau de administrator) este înregistrată.

- Evenimente de sistem: pornirea și oprirea serverului, conectarea la baza de date, erori interne de server (cod HTTP 500).

- **Perioadele de reținere a jurnalelor**

- În mediul de dezvoltare și demonstrație, jurnalele de audit sunt păstrate în consola serverului pe durata sesiunii de lucru. Nu există o cerință formală de arhivare pe termen lung. Pentru eventuale investigații, jurnalele pot fi redirecționate către un fișier (de exemplu, logs/audit.log) care poate fi păstrat pe durata desfășurării proiectului. La finalizarea proiectului, jurnalele pot fi șterse deoarece nu conțin date cu valoare arhivistică.

- **Rapoarte operaționale și de management**

- În versiunea curentă a sistemului, nu sunt generate rapoarte automate de audit. În caz de necesitate, un administrator poate inspecta manual jurnalele pentru a identifica evenimente suspecte. Pentru o versiune viitoare, s-ar putea implementa un endpoint /api/admin/audit care să returneze ultimele N evenimente de audit sub formă de JSON.

• 8.3 Piste de auditare a aplicațiilor

- **Implementarea pistelor de audit**

- Sistemul utilizează un middleware de logging care interceptează toate cererile HTTP înainte de a ajunge la controlere. Acest middleware înregistrează:
- Identificatorul unic al cererii (generat automat, de exemplu un UUID sau timestamp + număr secvențial)
- Timestamp-ul cererii (format ISO 8601)
- Adresa IP a clientului (din req.ip sau req.connection.remoteAddress)
- Metoda HTTP (GET, POST, PUT, DELETE)
- URL-ul complet al cererii
- User-id-ul utilizatorului autentificat (extras din req.session.userId, dacă există)
- După procesarea cererii, middleware-ul înregistrează și codul de stare HTTP al răspunsului și durata procesării în milisecunde.
- **Piste de audit pentru date critice specifice**
- Pentru accesul la datele critice (hash-urile parolelor), nu se implementează o pistă de audit separată deoarece aceste date nu sunt niciodată citite direct de aplicație. Singurul acces este prin bcrypt.compare, care nu returnează hash-ul ci doar un rezultat boolean.
- Pentru accesul la datele personale ale utilizatorilor (nume, email), pistele de audit sunt acoperite de logging-ul general al cererilor. Orice cerere către

/api/user/profile sau /api/admin/users va fi înregistrată, permițând identificarea eventualelor accesări neautorizate.

- Pentru modificarea datelor critice (schimbarea parolei, ștergerea contului), se adaugă înregistrări specifice cu nivel INFO, conținând detalii suplimentare precum tipul modificării și utilizatorul vizat.

8.4 Tabele standard pentru validarea câmpurilor de date

Tabele de validare implementate în cod

Sistemul nu utilizează tabele de validare în baza de date (lookup tables), deoarece toate valorile valide sunt definite direct în cod sau prin constrângeri CHECK în SQL. Următoarele validări sunt implementate:

Validarea criteriilor de optimizare: criteriile permise sunt definite într-o constantă în codul JavaScript: VALID_CRITERIA = ['time', 'cost', 'traffic', 'distance', 'eco']. Orice valoare care nu se află în această listă este respinsă.

Validarea tipurilor de utilizator: rolurile sunt definite prin coloana is_admin (boolean) în tabela users. Nu există alte roluri.

Validarea nivelului de congestie: se utilizează o constrângere CHECK în definiția tabelei traffic_data: CHECK (congestion_level BETWEEN 0 AND 100). Baza de date respinge automat valorile în afara acestui interval.

Validarea costului și duratei: constrângeri CHECK în tabela trips: CHECK (cost >= 0) și CHECK (duration >= 0).

Validarea formatului email: se face pe frontend (folosind atributul type="email" în HTML) și pe backend (folosind o expresie regulată simplă). Nu există o tabelă separată cu domenii de email valide.

Extinderea pentru versiuni viitoare

Dacă în viitor sistemul va necesita suport pentru mai multe tipuri de transport (auto, tren, avion) sau mai multe unități de măsură, s-ar putea adăuga tabele de validare în baza de date: de exemplu, tabela transport_types cu coloanele id și name, iar tabela trips ar avea o cheie străină către această tabelă.

8.5 Procese de verificare pentru adăugarea, ștergerea sau actualizarea datelor critice

Verificări pentru adăugarea datelor (CREATE)

La înregistrarea unui utilizator nou, sistemul efectuează următoarele verificări înainte de inserarea în baza de date:

Verificarea existenței emailului: se execută o interogare SELECT COUNT(*) FROM users WHERE email = $email. Dacă rezultatul este mai mare decât 0, se respinge înregistrarea cu mesajul "Acest email este deja folosit".

Validarea lungimii numelui: numele trebuie să aibă între 2 și 100 de caractere.

Validarea formatului emailului: folosind o expresie regulată.

Validarea lungimii parolei: parola trebuie să aibă minimum 6 caractere (validare pe backend, nu doar pe frontend).

După trecerea tuturor verificărilor, parola este criptată cu bcrypt și înregistrarea este inserată.

La salvarea unei rute în istoric, sistemul verifică că utilizatorul este autentificat (middleware isAuthenticated) și că ruta are toate câmpurile obligatorii completate (origin, destination, date). Verificările suplimentare (cost >= 0, duration >= 0) sunt delegate constrângerilor CHECk din baza de date.

Verificări pentru actualizarea datelor (UPDATE)

La actualizarea profilului unui utilizator (nume, email, parolă), sistemul efectuează următoarele verificări:

Dacă emailul este modificat, se verifică noul email să nu fie deja folosit de alt utilizator (SELECT COUNT(*) FROM users WHERE email = $newEmail AND id != $currentUserId).

Dacă parola este modificată, se verifică întâi parola veche (bcrypt.compare), apoi se criptează parola nouă.

Toate actualizările sunt efectuate într-o tranzacție pentru a preveni stări inconsistente.

La actualizarea unui utilizator de către administrator (de exemplu, setarea is_admin), sistemul verifică că utilizatorul care face cererea este administrator (middleware isAdmin). Nu se permite unui administrator să își revoce singur privilegiile de administrator (verificare suplimentară: dacă user_id-ul vizat este același cu administratorul care face cererea și acțiunea este setarea is_admin = FALSE, operația este respinsă pentru a preveni blocarea accidentală a accesului administrativ).

Verificări pentru ștergerea datelor (DELETE)

La ștergerea unei rute din istoric, sistemul verifică:

Că utilizatorul este autentificat (middleware isAuthenticated).

Că ruta aparține utilizatorului autentificat (interogarea DELETE include condiția AND user_id = $currentUserId). Dacă un utilizator încearcă să șteargă o rută care nu îi aparține, nu se șterge nicio înregistrare și se returnează codul 404 Not Found.

La ștergerea unui cont de utilizator de către administrator, sistemul verifică:

Că administratorul nu își șterge propriul cont (dacă user_id-ul vizat este același cu administratorul care face cererea, operația este respinsă). Ștergerea se face cu ON DELETE CASCADE, ceea ce înseamnă că toate călătoriile asociate utilizatorului vor fi șterse automat.

8.6 Capacitatea de a identifica informațiile de auditare

Informații colectate pentru fiecare eveniment de audit

Pentru fiecare cerere HTTP procesată, sistemul colectează și înregistrează următoarele informații, care permit identificarea completă a contextului evenimentului:

Identificarea utilizatorului: user_id extras din sesiune (dacă utilizatorul este autentificat) sau stringul "guest" pentru utilizatorii neautentificați. În cazul în care sesiunea nu conține user_id dar cookie-ul este prezent, se înregistrează "unauthenticated".

Identificarea terminalului de rețea: adresa IP a clientului, extrasă din req.ip sau req.connection.remoteAddress. În mediul de dezvoltare pe localhost, aceasta va fi

127.0.0.1 sau ::1. În cazul în care aplicația este accesată prin rețea, se va înregistra adresa IP reală.

Data și ora: timestamp-ul cererii în format ISO 8601 cu fus orar UTC, de exemplu "2026-04-26T10:30:00.000Z". Precizia este de o milisecundă.

Datele accesate sau modificate: URL-ul cererii (de exemplu, /api/trips/history) și metoda HTTP (GET, POST, PUT, DELETE). Pentru operațiile de modificare, se înregistrează suplimentar identificatorii resurselor afectate (de exemplu, id-ul călătoriei șterse). Corpul cererii nu este înregistrat pentru a evita expunerea datelor sensibile (cum ar fi parolele).

Exemplu de înregistrare de audit

În consola serverului, o înregistrare de audit tipică arată astfel:

[2026-04-26T10:30:00.123Z] [INFO] [AUTH] Login success - user_id=5, email=user@example.com, ip=127.0.0.1
[2026-04-26T10:31:00.456Z] [INFO] [ACCESS] GET /api/trips/history - user_id=5, ip=127.0.0.1, status=200, duration=45ms
[2026-04-26T10:32:00.789Z] [WARN] [AUTH] Login failed - email=wrong@example.com, ip=127.0.0.1, reason=invalid_password
[2026-04-26T10:33:00.012Z] [ERROR] [ACCESS] Unauthorized access attempt - user_id=3, ip=127.0.0.1, url=/api/admin/users, method=GET

Interogarea informațiilor de audit

În versiunea curentă, nu există o interfață dedicată pentru interogarea jurnalelor de audit. Un administrator poate inspecta direct consola serverului sau fișierul de log (dacă logging-ul este configurat să scrie într-un fișier). Pentru o versiune viitoare, s-ar putea

implementa un endpoint /api/admin/audit care să returneze jurnalele sub formă de JSON, cu posibilitatea de filtrare după user_id, dată, tip de eveniment.

Păstrarea confidențialității informațiilor de audit

Jurnalele de audit pot conține informații personale (emailuri, adrese IP). Accesul la aceste jurnale este restricționat la dezvoltator (studentul autor) și la administratorii sistemului. Jurnalele nu sunt distribuite public și sunt șterse la finalizarea proiectului.

Anexa A: Gestiunea modificărilor documentului

Prezenta anexă furnizează informații despre modul în care dezvoltarea și distribuția acestui document sunt controlate și urmărite. Documentul este gestionat ca parte a repository-ului Git al proiectului, iar versiunile sunt marcate prin tag-uri în sistemul de versionare.

Toate modificările aduse documentului sunt înregistrate în tabelul de mai jos, care include numărul versiunii, data modificării, autorul și o scurtă descriere a modificărilor efectuate.

Tabel 1 – Înregistrarea modificărilor asupreaa documentului curent

versiune	Data	Autorul/Deținătorul	Descriere
0.1	*15.04.2026*	*Copaci Iuliana-Cristina*	*Creare structură inițială a documentului de proiectare pe baza șablonului furnizat. Adăugare anteturi de secțiuni și cuprins.*
0.2	*22.04.2026*	*Copaci Iuliana-Cristina*	*Completare secțiunile principale: arhitectura sistemului, proiectarea bazei de date, scenarii de utilizare și proiectare de detaliu*
0.3	*24.04.2026*	*Copaci Iuliana-Cristina*	*Revizuire finală a întregului document. Corecturi gramaticale și de formatare. Verificare coerență între secțiuni. Document declarat final și pregătit pentru predare*

Anexa B: Acronime

Prezenta anexă furnizează lista acronimelor utilizate în cadrul acestui document, împreună cu forma completă a acestora. Acronimele sunt listate în ordine alfabetică.

Tabel 2 - Acronime

Acronim	Forma completă
API	Application Programming Interface (Interfață de Programare a Aplicațiilor)
BFS	Breadth-First Search (Căutare în lățime)
COTS	Commercial Off-The-Shelf (Produs comercial disponibil
CPU	Central Processing Unit (Unitate Centrală de Procesare
HTTP	Hypertext Transfer Protocol (Protocol de Transfer pentru Hipertext)
I/O	Input/Output (Intrare/Ieșire)
UI	User Interface (Interfață cu utilizatorul)
WAN	Wide Area Network (Rețea de Zonă Extinsă

Anexa C Documente la care se face referire

Prezenta anexă sintetizează relația acestui document cu alte documente relevante pentru proiectul TripSmart Agency. Sunt furnizate informații de identificare pentru toate documentele folosite pentru a ajunge la și/sau referite în acest document.

Relația dintre documente

Documentul de față (Documentul de proiectare) este strâns legat de Documentul de specificare a cerințelor (SRS). În timp ce SRS definește CE trebuie să facă sistemul (cerințele funcționale și non-funcționale), prezentul document definește CUM va fi construit sistemul pentru a îndeplini acele cerințe. Toate deciziile de proiectare luate în acest document sunt trasabile la cerințele specificate în SRS.

Șablonul de document de proiectare furnizat de cadrul didactic a fost utilizat ca structură de bază, fiind completat și adaptat specificului proiectului TripSmart Agency.

Documentațiile tehnice oficiale ale tehnologiilor utilizate au fost consultate pentru a asigura corectitudinea specificațiilor legate de versiuni, configurații și bune practici.

Lucrarea de licență va integra atât SRS-ul cât și prezentul document de proiectare, împreună cu capitolele de implementare, testare și concluzii, pentru a forma documentația completă a proiectului

Tabel 3 – Documente la care se face referire

Nume document	Locație sau URL	Dată emitere document
Documentul de specificare a cerințelor (SRS) - TripSmart Agency	*Local: /docs/SRS_TripSmart_final.docx*	*25.04.2026*
Șablon Document de proiectare - Laborator ASIPSI	*Furnizat de profesor, fișier local*	*03.02.2026*
Documentația oficială Node.js	*https://nodejs.org/en/docs/*	*17.03.2026*